

基于 C 语言的轻量化 DBMS 原型实验 指导书

DBMS_C 教学实践材料

2026 年 5 月 24 日

使用说明

本指导书面向数据库课程实践、计算机专业低年级学生以及 DBMS_C 项目阅读者。它围绕当前仓库中的真实 C 源码、测试入口、Trace 输出和图表组织实验内容，帮助读者逐步理解一个轻量化教学型 DBMS 原型的内部机制。

本指导书不是毕业论文，也不是工业级数据库手册。后续章节将遵循“先补充或确认真实 CTest，再撰写章节正文”的方式逐章完善；本轮仅完整整理第 0 章，并为第 1 章到第 12 章保留统一占位结构。

目录

第 0 章	引言与环境准备	1
0.1	编写目的	1
0.2	项目定位	1
0.3	学习主线	1
0.4	项目目录概览	1
0.5	环境准备	3
0.6	构建项目	3
0.7	运行测试与测试驱动的实验组织方式	4
0.8	运行系统	5
0.9	Trace 模式说明	6
0.10	如何阅读本指导书	6
0.11	本章小结	6
第 1 章	基础组件与工具库实验	7
1.1	实验目的	7
1.2	模块作用	7
1.3	相关源码文件	7
1.4	核心组件分类	7
1.5	在 DBMS 主链路中的位置	7
1.6	配套测试	9
1.7	关键代码讲解	10
1.8	运行与观察	12
1.9	思考题	12
第 2 章	文件与页式存储实验	13
2.1	实验目的	13
2.2	模块作用	13
2.3	相关源码文件	13
2.4	核心对象职责	14
2.5	文件块与内存页的关系	14
2.6	配套测试	14
2.7	关键代码讲解	15
2.8	运行与观察	18
2.9	思考题	19

第 3 章 日志管理实验	20
3.1 实验目的	20
3.2 模块作用	20
3.3 相关源码文件	20
3.4 核心对象职责	21
3.5 日志页与日志记录组织	21
3.6 配套测试	22
3.7 关键代码讲解	22
3.8 运行与观察	25
3.9 思考题	26
第 4 章 缓冲区管理实验	27
4.1 实验目的	27
4.2 模块作用	27
4.3 相关源码文件	27
4.4 核心对象关系	27
4.5 pin/unpin 机制	27
4.6 modified、dirty 与 flush 机制	29
4.7 配套测试	30
4.8 关键代码讲解	30
4.9 运行与观察	32
4.10 思考题	32
第 5 章 事务、并发与恢复实验	34
5.1 实验目的	34
5.2 模块作用	34
5.3 相关源码文件	34
5.4 Transaction 在系统中的位置	34
5.5 BufferList 的作用	36
5.6 事务基础读写与提交流程	36
5.7 配套测试	36
5.8 关键代码讲解	38
5.9 5B 并发锁基础实验	39
5.10 5C 恢复与回滚基础实验	41
5.11 运行与观察	43
5.12 后续小节 TODO	44
5.13 思考题	44
第 6 章 记录管理实验	45

6.1	实验目的	45
6.2	模块作用	45
6.3	相关源码文件	45
6.4	核心对象职责	45
6.5	Schema 与 Layout	47
6.6	RID 与记录位置	47
6.7	RecordPage 的 slot 生命周期	47
6.8	TableScan 顺序插入与读回	48
6.9	配套测试	48
6.10	关键代码讲解	49
6.11	运行与观察	51
6.12	思考题	51
第 7 章	元数据管理实验	52
7.1	实验目的	52
7.2	模块作用	52
7.3	相关源码文件	52
7.4	MetadataManager 与子管理器	52
7.5	系统目录表	54
7.6	CREATE TABLE 元数据登记过程	54
7.7	配套测试	55
7.8	关键代码讲解	56
7.9	运行与观察	57
7.10	思考题	58
第 8 章	SQL 解析实验	59
8.1	实验目的	59
8.2	SQL 解析在 DBMS_C 中的位置	59
8.3	相关源码文件	59
8.4	Lexer、Parser 与 PredParser 的职责	61
8.5	不同 SQL 命令的解析结果	61
8.6	配套测试	61
8.7	关键代码讲解	63
8.8	运行与观察	64
8.9	思考题	65
第 9 章	查询计划与扫描执行实验	66
9.1	实验目的	66
9.2	Plan/Scan 在 SQL 执行链路中的位置	66

9.3	模块作用	66
9.4	相关源码文件	67
9.5	Plan 与 Scan 的区别	67
9.6	SELECT 查询计划生成流程	67
9.7	配套测试	68
9.8	关键代码讲解	71
9.9	运行与观察	72
9.10	思考题	73
第 10 章	更新执行实验	74
10.1	实验目的	74
10.2	更新执行在系统中的位置	74
10.3	相关源码文件	74
10.4	PlannerExecuteUpdate 与 BasicUpdatePlanner	75
10.5	各类更新命令的执行路径	75
10.6	配套测试	77
10.7	关键代码讲解	79
10.8	运行与观察	81
10.9	思考题	81
第 11 章	SQL 执行过程 Trace 可视化实验	82
11.1	实验目的	82
11.2	Trace 模式定位	82
11.3	开启与关闭方式	82
11.4	Trace 标签说明	83
11.5	SELECT 查询 Trace 示例	83
11.6	更新命令 Trace 示例	84
11.7	事务命令 Trace 示例	85
11.8	人工验证步骤	85
11.9	演示注意事项	86
11.10	配套验证说明	86
11.11	关键代码讲解	87
11.12	运行与观察	88
11.13	思考题	89
第 12 章	综合实验	90
12.1	实验目的	90
12.2	相关源码文件	90
12.3	配套测试	90

12.4 实验内容	90
12.5 思考题	90

第 0 章 引言与环境准备

本章用于建立阅读 DBMS_C 的共同起点。读者在进入具体模块实验之前，需要先知道本项目的教学定位、源码目录如何组织、如何稳定构建和测试，以及后续章节为什么要采用“先测试、再写实验内容”的组织方式。

0.1 编写目的

本指导书面向计算机专业低年级学生、数据库课程实践和 DBMS_C 项目阅读者。它的目标不是让读者一次性掌握所有数据库实现细节，而是通过“源码阅读 + 测试运行 + Trace 观察 + 图表理解”的方式，逐步理解一个轻量化 DBMS 原型从 SQL 输入到文件存储访问的大致过程。

后续各章会围绕真实源码展开：先确认或补充可运行的测试，再把测试目标、运行命令、预期现象、观察点和对应源码位置写入章节正文。这样做的好处是，实验内容不只停留在文字说明，而是可以被当前工程实际验证。

0.2 项目定位

DBMS_C 是一个基于 C 语言实现的轻量化教学型 DBMS 原型，已经包含 SQL 解析、查询计划、扫描执行、记录管理、元数据管理、事务日志、缓冲区管理、文件页式存储等模块。它强调“可运行、可观察、可阅读”，适合作为数据库系统课程中的实验材料和源码阅读对象。

需要注意的是，本项目不是完整工业级数据库系统。它不会覆盖工业数据库中的全部 SQL 标准、优化器规则、并发事务隔离级别、存储引擎能力和高性能工程化细节。本指导书也不是论文式说明文档，而是配套 DBMS_C 源码的实验指导书。

0.3 学习主线

阅读 DBMS_C 时，可以沿着从用户输入向底层存储逐步下沉的方向理解系统。图 0-1 不是完整函数调用图，而是一张学习路线图：上层更接近 SQL 和用户观察结果，下层更接近数据页、文件和事务支撑。

图 0-1 中，各层的作用可以这样理解：SQL 输入层负责接收用户命令；SQL 解析层把文本转换为内部数据对象；查询计划层决定如何组合表扫描、选择和投影；Scan 扫描执行层以统一接口向上提供记录；记录访问层负责表记录的定位与读写；元数据层保存表结构、视图和索引相关信息；事务与日志层为更新和恢复提供支撑；缓冲区层管理内存中的页；文件与页式存储层最终负责磁盘文件和页面读写。Trace 模式不改变执行语义，只帮助读者观察这些阶段的大致过程。

0.4 项目目录概览

DBMS_C 的目录不是按论文章节划分，而是按数据库执行链路和模块职责组织。表 0-1 根据当前仓库中的真实目录整理，便于读者在后续实验中快速定位源码。

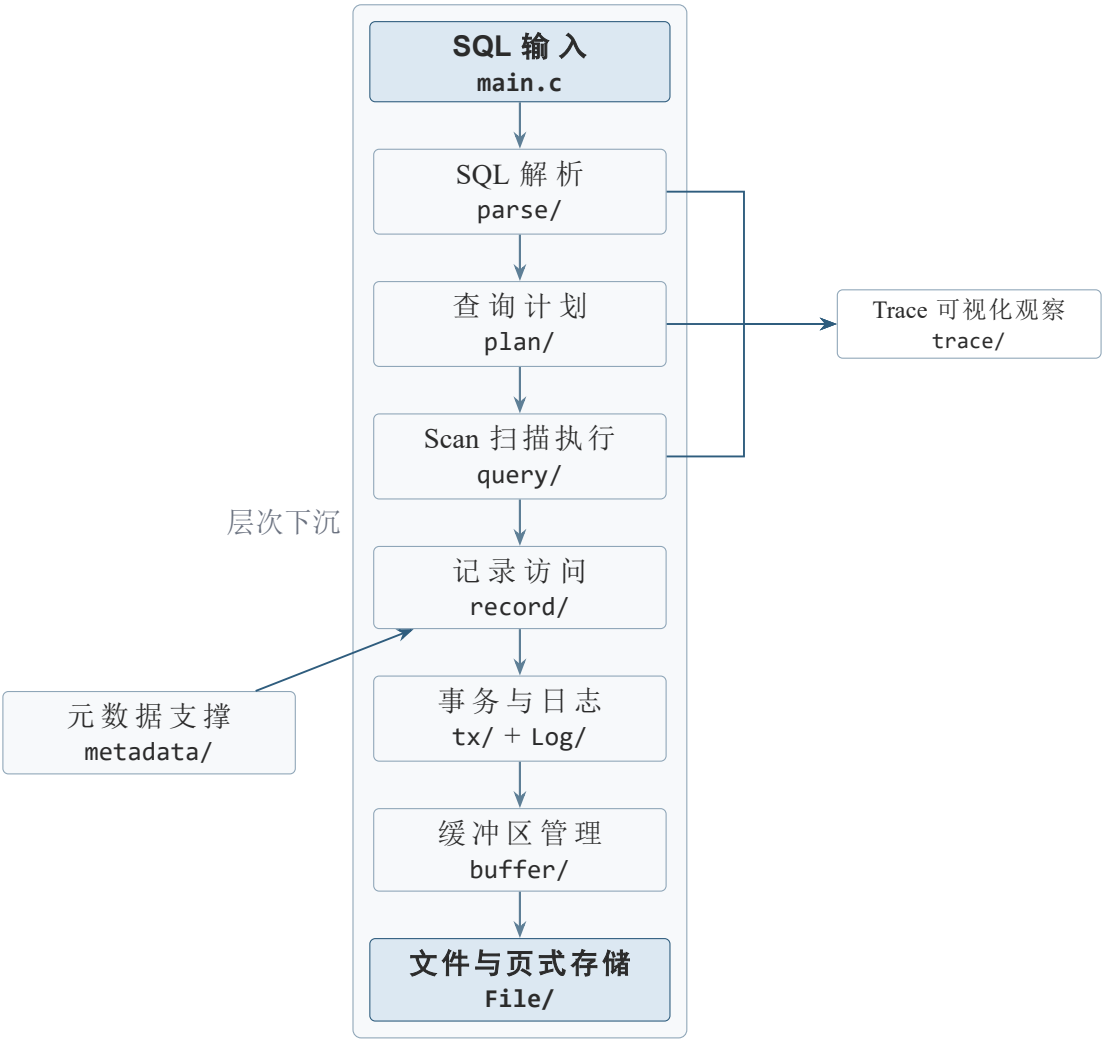


图 0-1 DBMS_C 学习主线图

表 0-1 DBMS_C 项目目录职责表

目录或文件	主要职责
Lib/	基础工具库，包括字符串、字节缓冲、列表、映射、向量、分词辅助等通用组件。
File/	文件块、页对象和文件管理，是理解页式存储的入口。
Log/	日志管理相关实现，为事务提交、回滚和恢复提供基础。
buffer/	缓冲区管理和 LRU 替换策略，负责内存页与磁盘页之间的协调。
tx/	事务、并发控制、恢复管理以及事务持有的缓冲区列表。
record/	Schema、Layout、RID、RecordPage、TableScan 等记录层结构。
metadata/	表、字段、视图、索引和统计信息等元数据管理。
parse/	Lexer、Parser 以及各类 SQL 命令的数据对象。
plan/	查询计划和更新计划的构造入口。
query/	Scan 接口、表达式、谓词、选择、投影和笛卡尔积扫描等执行层对象。
index/	当前仓库中的哈希索引相关实现。
DBMS/	SimpleDB 初始化入口，连接文件、日志、缓冲区、元数据和规划器。
main.c	CLI 入口，负责读取命令、分发 SQL、执行事务命令和输出结果。
test/	基于 CMocka 的已有测试文件。
docs/	说明文档、Trace 文档和本实验指导书。

推荐阅读顺序是: Lib/ → File/ → Log/ → buffer/ → tx/ → record/ → metadata/ → parse/ → plan/ 与 query/ → 更新执行 → Trace → 综合实验。这个顺序先建立底层对象概念，再理解上层 SQL 如何使用这些能力。

0.5 环境准备

本指导书默认在 Windows 环境下进行实验。当前项目的稳定构建方式依赖 CMake、MinGW 编译器和 mingw32-make。编译本指导书本身还需要 XeLaTeX。

建议准备以下环境：

- Windows 操作系统；
- MinGW 或等价的 GCC 工具链，并确保 mingw32-make 可用；
- CMake；
- XeLaTeX，用于编译本指导书；
- 可选编辑器：VS Code、CLion、Trae 或 Codex。

如果不同机器上的工具链路径不同，应优先保证命令行中能直接执行 cmake、mingw32-make 和 xelatex。后续章节不会假定某一个 IDE 的专有功能。

0.6 构建项目

当前 Windows/Codex 环境下，DBMS_C 已验证的稳定构建命令如表 0-2 所示。请先进入项目根目录，再进行配置和编译。

表 0-2 构建与测试命令表

命令	作用
cd D:\code\DBMS\NewDBMS\DBMS_C	进入 DBMS_C 项目目录。
cmake -S . -B build	根据当前 CMakeLists.txt 配置构建目录。
mingw32-make -C build -j8 SHELL=cmd.exe	使用 mingw32-make 编译项目，并显式覆盖 Makefile 中的 shell。
ctest --test-dir build --output-on-failure	运行已注册测试，并在失败时输出详细信息。
bin\NewDBMS.exe	启动交互式 DBMS 程序。

代码 0-1 稳定构建命令

```
cd D:\code\DBMS\NewDBMS\DBMS_C
cmake -S . -B build
mingw32-make -C build -j8 SHELL=cmd.exe
```

这里使用 mingw32-make -C build -j8 SHELL=cmd.exe，是因为当前构建目录在 Windows 下使用 Makefile 时，默认 shell 可能指向 /bin/sh。在 Codex/Trae 这类环境中，调用 Git/MSYS 的 shell 曾经触发过构建阶段问题。因此，本指导书把 mingw32-make 并覆盖 SHELL=cmd.exe 作为当前稳定主命令，不把 cmake --build build 作为默认主命令。

0.7 运行测试与测试驱动的实验组织方式

项目测试通过 CTest 统一运行：

代码 0-2 运行已有测试

```
ctest --test-dir build --output-on-failure
```

ctest 用于确认已有模块测试是否通过。后续每章新增或调整测试后，都要重新运行完整 CTest，确保没有破坏已有测试。如果新增测试，测试数量可能增加；如果测试失败，应优先查看第一条真实失败信息，而不是同时修改多个无关位置。

从第 1 章开始，本指导书采用测试驱动的实验组织方式：

- 1. 每个模块实验在正式撰写章节前，先补充或确认一个真实可运行的测试。
- 2. 测试必须基于当前仓库真实 API，不能编造不存在的函数、结构体或路径。
- 3. 测试通过后，章节中再写入测试文件路径、测试目标、运行命令、预期结果、观察点和对应源码位置。
- 4. 文档中不能写不存在或没有跑通的测试。

- 5. 如果某章暂时没有测试，就只能写“后续补充”，不能伪造测试结果。
- 6. 每新增一个测试后都要运行完整 CTest。
- 7. 第 0 章本身不新增模块测试，只记录当前工程的基线构建与测试方法。

表 0-3 给出后续章节与测试策略的对应关系。当前状态均为 TODO，表示需要在后续轮次中先处理测试，再完善章节正文。

表 0-3 后续章节与测试策略对照表

章节	测试策略
第 1 章基础组件与工具库实验	TODO：先补充或确认 Lib 相关 CTest，再撰写章节内容。
第 2 章文件与页式存储实验	TODO：先补充或确认 File/Page 相关 CTest，再撰写章节内容。
第 3 章日志管理实验	TODO：先补充或确认 Log 相关 CTest，再撰写章节内容。
第 4 章缓冲区管理实验	TODO：先补充或确认 buffer 相关 CTest，再撰写章节内容。
第 5 章事务、并发与恢复实验	TODO：先补充或确认 tx/recovery 相关 CTest，再撰写章节内容。
第 6 章记录管理实验	TODO：先补充或确认 record 相关 CTest，再撰写章节内容。
第 7 章元数据管理实验	TODO：先补充或确认 metadata 相关 CTest，再撰写章节内容。
第 8 章 SQL 解析实验	TODO：先补充或确认 parse 相关 CTest，再撰写章节内容。
第 9 章查询计划与扫描执行实验	TODO：先补充或确认 plan/query 相关 CTest，再撰写章节内容。
第 10 章更新执行实验	TODO：先补充或确认更新执行相关 CTest，再撰写章节内容。
第 11 章 SQL 执行过程 Trace 可视化实验	TODO：先确认 Trace 行为和可观察示例，再撰写章节内容。
第 12 章综合实验	TODO：在前序实验稳定后设计综合测试和综合任务。

0.8 运行系统

构建完成后,主程序位于项目根目录下的 bin/NewDBMS.exe。在 Windows PowerShell 中可以这样启动：

代码 0-3 启动 DBMS_C

```
cd D:\code\DBMS\NewDBMS\DBMS_C
bin\NewDBMS.exe
```

程序启动后会进入交互式 SQL 提示符。读者可以输入 SQL 语句并以分号结束，也可以输入 commit、rollback、trace on、trace off、exit 等元命令。输入 exit 会退

出系统；按当前程序逻辑，退出阶段会提交当前事务。因此，如果 Trace 已开启，读者可能在关闭数据库时看到一段退出阶段的 COMMIT Trace。

0.9 Trace 模式说明

Trace 模式用于观察 SQL 在 DBMS_C 内部的大致执行过程，包括解析、计划、扫描、更新、事务和输出等阶段。它的定位是 CLI 教学可视化，不是 GUI，也不改变普通 SQL 执行语义。

当前仓库中已经有 docs/trace_mode.md，用于记录 Trace 模式的开启方式、示例 SQL 和当前限制。本指导书第 11 章会进一步围绕 Trace 设计实验；第 0 章只要求读者知道 Trace 是一种观察执行链路的辅助工具。

0.10 如何阅读本指导书

建议按以下方式使用本指导书：

1. 先阅读每章实验目的，明确本章要理解的问题。
2. 再查看相关源码文件，确认结构体、函数和目录都来自真实仓库。
3. 再运行本章配套测试，观察测试是否通过以及输出是否符合预期。
4. 对照图、表、代码和 Trace 输出理解执行链路。
5. 最后完成思考题，把模块放回 DBMS 的整体执行路径中理解。

如果某章暂时只有 TODO，占位内容表示该章还没有完成“先测试、再写章节”的准备工作。不要把 TODO 当成已完成实验，也不要根据 TODO 推断源码中已经存在某个未验证接口。

0.11 本章小结

本章完成了指导书的起步准备：明确了 DBMS_C 的教学型原型定位，给出了从 SQL 输入向底层存储下沉的学习主线，整理了真实项目目录职责，说明了 Windows 环境下稳定构建、测试和运行系统的方法，并规定了后续章节采用测试驱动的实验组织方式。

从下一章开始，每一章都应先围绕对应模块补充或确认真实可运行的 CTest，再把测试入口、源码位置和观察结果写入章节正文。下一步建议从第 1 章基础组件与工具库实验开始。

第 1 章 基础组件与工具库实验

1.1 实验目的

本章用于理解 DBMS_C 为什么需要自定义基础组件。C 语言本身没有内建字符串对象、动态数组、链表、映射容器和统一字节缓冲区；如果每个上层模块都直接操作裸指针和裸数组，SQL 解析、记录布局、页式存储和元数据管理会很快变得难以阅读和验证。

本章实验的目标是先确认 Lib/ 中若干基础组件可以独立工作，再观察它们如何支撑上层 DBMS 模块。实验范围以当前真实 API 和已经跑通的 test/Lib/LibBasicTest.c 为准，不扩展未验证接口。

1.2 模块作用

Lib/ 为上层模块提供字符串、动态容器、字节缓冲和词法切分等基础能力。它不是一个独立数据库模块，而是许多数据库对象共同依赖的工具层：表名、字段名和文件名依赖字符串组件；字段列表、表列表和谓词项依赖容器组件；页内数据编码依赖字节缓冲组件；SQL 解析入口依赖词法切分组件。

这些基础组件越稳定，上层模块越容易被单独测试。反过来说，如果基础组件存在内存泄漏、越界读写或比较规则错误，问题可能会在 Parser、Record、Metadata、Buffer 等更高层才表现出来，定位会更加困难。

1.3 相关源码文件

表 1-1 列出了本章展开的真实文件。这里的“展开”表示本章会描述其基本职责，并在配套测试中至少覆盖其中一部分稳定 API。

本章不展开 rwlock.h、BlockLockManager.c/.h、RBT 和 Trie。其中，RBT 作为 CMap 的内部依赖出现，暂不作为独立实验目标；Trie 和其他工具文件后续只有在确认主链路用途和测试入口后再展开。

1.4 核心组件分类

从实验角度看，Lib/ 可以先按用途分为五类。表 1-2 给出了本章的分类方式和对对应观察点。

这种分类不是源码目录的全部内容，而是本章实验的最小切入点。先确认这些组件独立可用，再阅读上层模块会更顺畅。

1.5 在 DBMS 主链路中的位置

图 1-1 展示了 Lib/ 对上层模块的支撑关系。图中不是函数调用顺序，而是“基础能力被谁使用”的关系。

CString 支撑 SQL 文本、字段名、表名、文件名等字符串对象；CVector、CList

表 1-1 本章涉及的 Lib 源码文件

文件	本章关注点
Lib/CString.h, Lib/CString.c	字符串创建、追加、获取内容、长度和比较。
Lib/CVector.h, Lib/CVector.c	按元素大小管理动态数组，支持插入、读取、查找和设置。
Lib/CList.h, Lib/CList.c	单链表容器，支持追加、按下标读取、contains、浅拷贝和深拷贝等行为。
Lib/CMap.h, Lib/CMap.c	基于红黑树包装的映射容器，本章只验证最小插入、查找和更新。
Lib/map.h	元数据等模块使用的宏式 map，本章只作为目录职责说明，不新增单独测试。
Lib/ByteBuffer.h, Lib/ByteBuffer.c	连续字节缓冲区，支持 int、long、bytes 等读写。
Lib/StreamTokenizer.h, Lib/StreamTokenizer.c	SQL 文本的轻量 token 切分工具。
Lib/Error.h, Lib/Error.c	旧错误对象，当前已标记为 legacy/deprecated，本章只验证默认初始化。

表 1-2 Lib 组件分类表

分类	代表文件	观察点
字符串组件	CString.h/.c	是否能保存、追加和比较 SQL 文本中的名称。
容器组件	CVector.h/.c, CList.h/.c, CMap.h/.c	是否能保存字段列表、表列表、谓词项和键值映射。
字节缓冲组件	ByteBuffer.h/.c	是否能按小端方式把整数编码到连续字节区并读回。
词法切分组件	StreamTokenizer.h/.c	是否能把 SQL 文本分成 WORD、NUMBER、STRING、DELIM 等 token。
错误处理辅助组件	Error.h/.c	旧错误结构体是否能默认初始化；新代码优先使用 error/DBError.h。

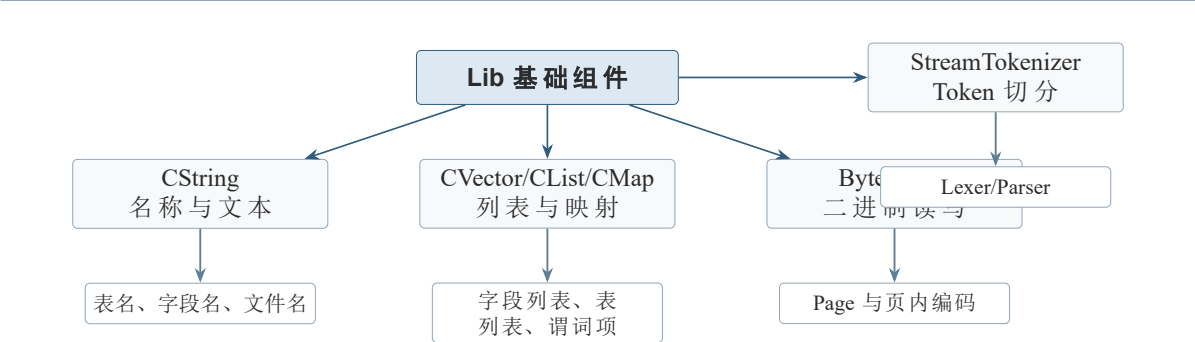


图 1-1 Lib 支撑 DBMS 上层模块关系图

和 CMap 支撑字段列表、表列表、谓词项、元数据组织等结构；ByteBuffer 支撑页内字节读写和底层数据编码；StreamTokenizer 支撑 SQL 解析前的词法切分。

图 1-2 用一个简单的整数写入示意 ByteBuffer 的基本思想：写入整数时，缓冲区从当前 position 开始连续写入 4 个字节；bufferFlip 会把 limit 设为已经写入的位置，并把 position 归零，随后即可按相同布局读回。

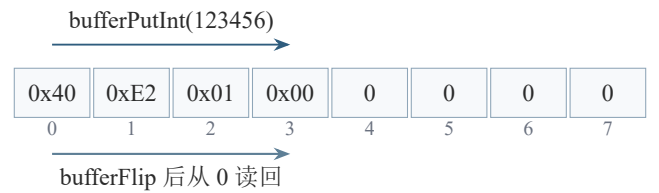


图 1-2 ByteBuffer 字节布局示意图

图 1-3 展示了 StreamTokenizer 对简单 SQL 片段的切分结果。它只负责把字符流分段，并记录 token 类型、起始位置、长度和整数值；更高层的语法含义仍由 Lexer/Parser 处理。



图 1-3 StreamTokenizer token 切分示意图

1.6 配套测试

本章新增并跑通的配套测试文件是：

- test/Lib/LibBasicTest.c
- 测试名称为 LibBasicTest。它验证了以下最小行为：
- CString：创建、追加、获取内容、长度和比较。
- CVector：插入整数、读取元素、查询大小、查找和设置。
- CList：追加、按下标读取、contains。
- CMap：插入、查找、通过 CMapPut 更新。
- ByteBuffer：写入 int、flip 后读回。
- StreamTokenizer：对简单 SQL 文本切分 WORD、DELIM、NUMBER。

- **Error:** 旧错误对象默认初始化。

运行本章测试可以使用 CTest 的正则过滤：

代码 1-1 运行 Lib 相关测试

```
ctest --test-dir build --output-on-failure -R Lib
```

完整回归测试仍然使用：

代码 1-2 运行完整测试

```
ctest --test-dir build --output-on-failure
```

本轮新增测试后，完整 CTest 已从 13 个测试增加到 14 个测试，并通过 14/14。

1.7 关键代码讲解

第一段代码来自 Lib/CString.c 的追加逻辑。它先计算追加后的长度，再确保容量足够，最后把新内容接到已有字符串尾部。

代码 1-3 CStringAppendCStr 的核心逻辑

```
1 void CStringAppendCStr(CString* cs, const char* str) {  
2     if (!cs || !str) return;  
3  
4     size_t appendLen = strlen(str);  
5     size_t newLen = cs->length + appendLen;  
6  
7     if (!ensureCapacity(cs, newLen + 1)) return;  
8  
9     strcat(cs->data + cs->length, str);  
10    cs->length = newLen;  
11 }
```

这段代码说明 CString 不只是保存一个 char*，还维护当前长度和容量。上层模块使用 CStringGetPtr 读取内容时，不需要关心扩容细节。

第二段代码来自 Lib/CVector.c 的尾插逻辑。它根据元素大小计算目标地址，并根据是否提供 Copy 回调决定复制方式。

代码 1-4 CVectorPushBack 的核心逻辑

```
1 void CVectorPushBack(CVector *vec, const void *value) {
2     if (!vec || !value) {
3         return;
4     }
5
6     if (vec->size + 1 > vec->capacity && !CVectorGrow(vec, vec->size + 1)) {
7         return;
8     }
9
10    void *dest = (char *)vec->data + vec->size * vec->elem_size;
11    if (vec->Copy) {
12        vec->Copy(dest, value);
13    } else {
14        memcpy(dest, value, vec->elem_size);
15    }
16    vec->size++;
17 }
```

这段代码体现了 C 语言中泛型容器的常见做法：容器不知道元素类型，只知道元素大小和可选回调函数。

第三段代码来自 Lib/ByteBuffer.c。写入整数时，代码按字节写入并移动 position；读出时按同样的字节布局恢复整数。

代码 1-5 ByteBuffer 写入 int 的核心逻辑

```
1 ByteBufferError bufferPutInt(ByteBuffer* buffer, int32_t data){
2     if (!buffer || !buffer->data) return BYTEBUFFER_ERROR_NULL;
3     if (buffer->position + 4 > buffer->limit) return BYTEBUFFER_ERROR_BOUNDS;
4     buffer->data[buffer->position++] = data & 255;
5     buffer->data[buffer->position++] = (data >> 8) & 255;
6     buffer->data[buffer->position++] = (data >> 16) & 255;
7     buffer->data[buffer->position++] = (data >> 24) & 255;
8     return BYTEBUFFER_OK;
9 }
```

这段代码和 File/Page 模块关系很近。后续学习页式存储时，会看到页内整数和字符串最终都需要落到字节数组中。

第四段代码来自 Lib/StreamTokenizer.c。它识别由字母或下划线开头的 WORD token，并通过 start 和 length 描述 token 内容。

代码 1-6 StreamTokenizer 识别 WORD 的核心逻辑

```
1 if (isalpha((unsigned char)c) || c == '_') {  
2     while (isalnum((unsigned char)*st->pos) || *st->pos == '_') st->pos++;  
3     st->length = (int)(st->pos - st->start);  
4     return (st->type = STT_WORD);  
5 }
```

这说明 StreamTokenizer 只做轻量词法切分，不判断 select、from、where 的语法位置。语法规则会在更高层解析器中处理。

1.8 运行与观察

本章建议先运行 Lib 相关测试：

代码 1-7 观察 Lib 测试

```
ctest --test-dir build --output-on-failure -R Lib
```

观察时重点关注四件事：

- 基础组件是否能在不启动数据库主程序的情况下独立工作。
- 字符串和容器是否能够支撑字段名、表名、列表和映射等上层数据结构。
- ByteBuffer 是否能完成基本二进制写入、翻转和读回。
- StreamTokenizer 是否能把 SQL 文本切成可供解析器继续处理的 token。

如果本章测试失败，应先看 LibBasicTest 的第一条失败信息，再回到对应头文件和实现文件确认真实 API 行为。

1.9 思考题

1. 为什么 C 语言项目需要自己封装 CString 和 CVector？
2. ByteBuffer 与后续 File/Page 模块之间是什么关系？
3. StreamTokenizer 为什么可以放在 Lib/，而不是直接写进 parse/？
4. 如果基础组件存在内存泄漏，会影响哪些上层模块？请结合表名、字段列表、页数据和 SQL 解析举例说明。

第 2 章 文件与页式存储实验

2.1 实验目的

本章用于理解 DBMS_C 如何把上层记录访问落实到底层文件块和内存页。数据库表、日志和记录并不是直接以“对象”的形式写入磁盘，而是最终落到固定大小的文件块上；程序运行时，又需要把这些文件块读入内存中的页缓冲区，再按整数、字符串或原始字节进行解释。

本章实验只关注 File/ 目录中的最小闭环：BlockID 标识一个文件块，Page 管理内存页内容，FileManager 负责把页写入文件块或从文件块读回页。配套测试已经注册为 FileManagerBasicTest，并通过完整 CTest 验证。

2.2 模块作用

File/ 为上层 buffer/、record/、Log/ 等模块提供块级文件访问能力。它不理解表结构、字段类型或事务语义，只负责三个基础问题：目标块在哪里、页内字节如何读写、内存页如何和本地文件块交换数据。

这种分层让上层模块可以把“读第几个块”“写一页数据”“追加一个新块”作为稳定接口使用，而不必在每个模块里重复处理文件路径、偏移量、fread、fwrite 和块大小计算。

2.3 相关源码文件

表 2-1 列出本章展开的真实源码文件。DBMS_C 当前没有在 File/ 中定义单独的全局 BLOCK_SIZE 常量，块大小由 FileManagerInit(dbDirectoryName, blockSize) 传入，页大小由 PageInit(size) 传入；本章测试使用 TEST_BLOCK_SIZE 400 保持二者一致。

表 2-1 File/Page 相关源码文件

文件	本章关注点
File/BlockID.h	声明文件名和块号组成的块标识结构及访问函数。
File/BlockID.c	实现 BlockIDInit、BlockIDEqual、BlockID2CString 等块标识操作。
File/Page.h	声明页对象以及 int、short、long、string、bytes 的页内读写接口。
File/Page.c	基于 ByteBuffer 实现页内偏移读写和字符串长度前缀编码。
File/FileManager.h	声明文件管理器结构和 read、write、length、append 等块级操作。
File/FileManager.c	负责数据库目录创建、文件句柄缓存、按块偏移读写和追加空块。

从阅读顺序上，建议先看 `BlockID.h/.c`，理解“文件名 + 块号”如何定位块；再看 `Page.h/.c`，理解页内数据布局；最后看 `FileManager.h/.c`，把页和磁盘文件联系起来。

2.4 核心对象职责

图 2-1 展示了三个核心对象之间的关系。它们不是并列工具，而是一条从“定位”到“内存表示”再到“文件读写”的小链路。

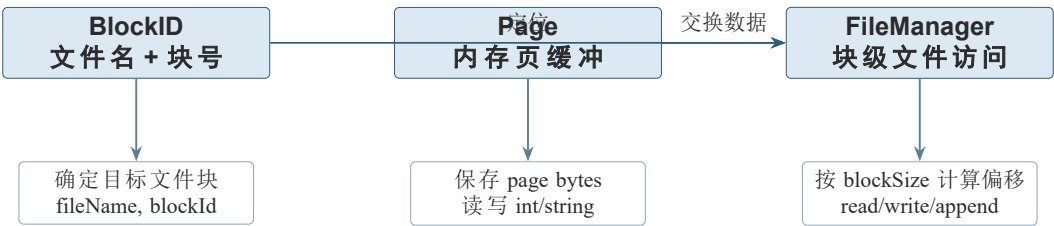


图 2-1 BlockID、Page 与 FileManager 的职责关系

`BlockID` 保存文件名和块号，用于回答“这次读写的是哪个文件的第几个块”。`Page` 是内存中的页级字节缓冲，对上提供 `PageSetInt`、`PageGetInt`、`PageSetString`、`PageGetString` 等接口。`FileManager` 保存数据库目录名、块大小和文件句柄缓存，负责把 `Page` 中的字节写到指定 `BlockID`，或把指定块读回到另一个 `Page`。

2.5 文件块与内存页的关系

数据库底层通常按块或页读写文件。`DBMS_C` 中，文件块是磁盘文件上的一段连续字节，`Page` 是内存中的同样大小的字节缓冲。二者的大小必须在一次读写中保持一致，否则页内偏移和文件块偏移就无法对应。

图 2-2 展示了第 n 个文件块与一个内存 `Page` 的对应关系。实际读写时，`FileManager` 使用 $\text{blockSize} * \text{blockId}$ 计算文件偏移，再用 `fread` 或 `fwrite` 交换整页数据。

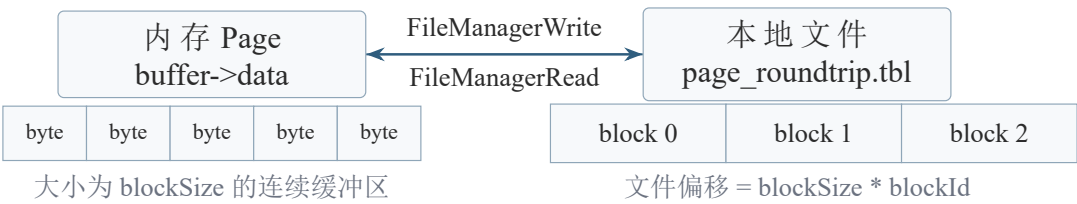


图 2-2 文件块与内存 Page 的对应关系

上层记录和日志最终都依赖这一层完成持久化：记录模块负责解释页内槽位和字段，日志模块负责组织日志记录，而 `FileManager` 只负责把已经组织好的页字节写入对应文件块。

2.6 配套测试

本章新增并注册了配套测试文件：

- `test/File/FileManagerBasicTest.c`

该测试使用 CMocka，测试名为 FileManagerBasicTest。它覆盖以下最小行为：

表 2-2 FileManagerBasicTest 覆盖内容

测试函数	验证目标
test_page_write_read_int	PageSetInt 写入整数后，PageGetInt 能按相同偏移读回。
test_page_write_read_string	PageSetString 写入字符串后，PageGetString 能读回相同内容。
test_block_id_create_compare_and_string	BlockID 能保存文件名和块号，并支持比较和字符串化。
test_file_manager_write_and_read_block	一个 Page 写入块后，另一个 Page 能从同一块读回 int 和 string。
test_file_manager_append_block	FileManagerAppend 能追加块，并让文件长度随块数增长。

测试使用独立测试数据库目录，并在写读块和追加块用例中按进程号生成目录名，避免重复运行 CTest 时被上一次 Windows 文件句柄释放时机影响。测试会尝试在用例前后清理测试目录；如果 Windows 暂时保留目录句柄，后续测试也不会复用同名目录。

2.7 关键代码讲解

代码 2.7 来自 File/BlockID.c。可以看到，BlockID 并不保存文件句柄，只保存文件名副本和块号。比较两个块是否相同，也必须同时比较块号和文件名。

代码 2-1 BlockID 创建与比较

```
1 BlockID *BlockIDInit(CString *name, int id){
2     if (!name)
3         return NULL;
4     BlockID *b = (BlockID *)malloc(sizeof(BlockID));
5     b->fileName = CStringCreateFromCString(name);
6     b->BlockID = id;
7     return b;
8 }
9
10 bool BlockIDEqual(BlockID *b1, BlockID *b2){
11     return BlockIDGetBlockID(b1) == BlockIDGetBlockID(b2)
12     && CStringEqual(BlockIDGetFileName(b1),BlockIDGetFileName(b2));
13 }
```

代码 2.7 来自 File/Page.c。Page 本身把实际字节操作交给 ByteBuffer；字符串写入时先写长度，再写字符内容，读回时按照同样布局恢复 CString。

代码 2-2 Page 中 int 与 string 的读写

```

1  int PageGetInt(Page *p, int position) {
2      int output;
3      bufferGetIntPosition(p->buffer, position, &output);
4      return output;
5  }
6
7  void PageSetInt(Page *p, int position, int data) {
8      bufferPutIntPosition(p->buffer, position, data);
9  }
10
11 CString *PageGetString(Page *p, int position) {
12     p->buffer->position = position;
13     int length = 0;
14     bufferGetInt(p->buffer, &length);
15     if (length < 0) {
16         return NULL;
17     }
18     char *str = (char*)malloc(length + 1);
19     bufferGetBytesPosition(p->buffer, position + sizeof(int32_t), str, length);
20     str[length] = '\0';
21     CString *cString = CStringCreateFromCStr(str);
22     return cString;
23 }
24
25 void PageSetString(Page *p, int position, const CString *cs) {
26     int length = (int)cs->length;
27     p->buffer->position = position;
28     bufferPutInt(p->buffer, length);
29     bufferPutBytes(p->buffer, (uint8_t*)cs->data, length);
30 }

```

代码 2.7 来自 File/FileManager.c。FileManagerRead 和 FileManagerWrite 的核心都是先通过 BlockID 找到文件，再根据块号计算偏移。读不存在的块时，当前实现会把目标 Page 填 0，这让上层可以得到一个空页。

代码 2-3 FileManager 按块读写

```

1  void FileManagerRead(FileManager *fm, BlockID *blockId, Page *page) {
2      FILE *f = FileManagerGetFile(fm, blockId->fileName);
3      if (f == NULL) {
4          perror("File not found");
5          return;
6      }
7      long target_offset = (long)fm->blockSize * BlockIDGetBlockID(blockId);
8      fseek(f, 0, SEEK_END);

```

```

9     long file_size = ftell(f);
10    if (target_offset >= file_size) {
11        memset(page->buffer->data, 0, fm->blockSize);
12        return;
13    }
14
15    fseek(f, fm->blockSize * BlockIDGetBlockID(blockId), SEEK_SET);
16    size_t bytesRead = fread(page->buffer->data, 1, fm->blockSize, f);
17    if (bytesRead < fm->blockSize) {
18        memset(page->buffer->data + bytesRead, 0,
19              fm->blockSize - bytesRead);
20    }
21 }
22
23 void FileManagerWrite(FileManager *fm, BlockID *blockId, Page *page) {
24     FILE *f = FileManagerGetFile(fm, blockId->fileName);
25     if (f == NULL) {
26         perror("File not found");
27         return;
28     }
29     fseek(f, fm->blockSize * BlockIDGetBlockID(blockId), SEEK_SET);
30     size_t bytesRead = fwrite(page->buffer->data, 1, fm->blockSize, f);
31     if (bytesRead != fm->blockSize) {
32         perror("Error! . Failed to read the complete block");
33     }
34     fflush(f);
35 }

```

代码 2.7 展示了追加块的关键逻辑。它先用 `FileManagerLength` 得到新块号，再写入一个全 0 缓冲区，从而扩展文件长度。

代码 2-4 FileManager 追加新块

```

1 int FileManagerLength(FileManager *fm, CString *filename) {
2     FILE *f = FileManagerGetFile(fm, filename);
3     fseek(f, 0, SEEK_END);
4     return (int)(ftell(f) / fm->blockSize);
5 }
6
7 BlockID *FileManagerAppend(FileManager *fm, CString *filename) {
8     int newBlkNum = FileManagerLength(fm, filename);
9     char *buffer = calloc(1, fm->blockSize);
10    BlockID *blk = BlockIDInit(CStringCreateFromCString(filename), newBlkNum);
11    FILE *f = FileManagerGetFile(fm, filename);
12    fseek(f, (long)blk->BlockID * fm->blockSize, SEEK_SET);
13    size_t bytesWritten = fwrite(buffer, 1, fm->blockSize, f);
14    fflush(f);

```

```

15     if (bytesWritten != fm->blockSize) {
16         perror("Failed to append block");
17     }
18     free(buffer);
19
20     return blk;
21 }

```

2.8 运行与观察

本章测试已经接入根目录 `CMakeLists.txt`。在 `D:\textbackslash code\textbackslash DBMS\textbackslash NewDBMS\textbackslash DBMS_C` 下使用固定命令构建和运行：

代码 2-5 构建并运行 File 相关测试

```

1 cmake -S . -B build
2 mingw32-make -C build -j8 SHELL=cmd.exe
3 ctest --test-dir build --output-on-failure -R File

```

如果只想运行本章新增测试，可以使用：

代码 2-6 只运行 FileManagerBasicTest

```

1 ctest --test-dir build --output-on-failure -R FileManagerBasicTest

```

图 2-3 概括了本章最关键的写入再读回过程。测试先在一个 Page 中写入整数和字符串，再调用 `FileManagerWrite` 写入文件块，随后用另一个 Page 调用 `FileManagerRead` 读回，并断言内容一致。

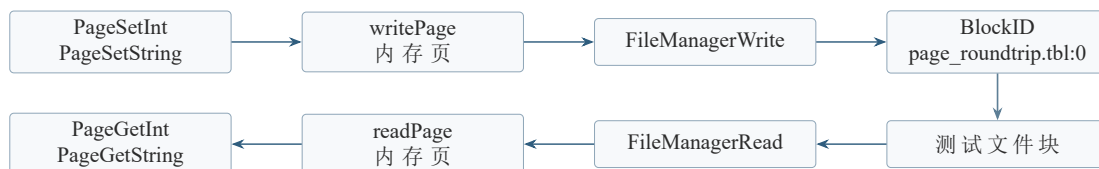


图 2-3 Page 写入、FileManager 落盘、再读回的调用链

运行本章实验时，建议重点观察四件事：第一，Page 写入的数据是否能在同一页中读回；第二，`FileManagerWrite` 写入块后，另一个 Page 是否能从同一块读回相同内容；第三，BlockID 如何用文件名和块号确定目标位置；第四，这一层如何为后续 `BufferManager` 提供“读块、写块、追加块”的底座。

2.9 思考题

1. 为什么数据库通常按块或页读写文件，而不是每次只读写一个字段？
2. **Page** 为什么要提供 **int** 和 **string** 的读写接口？这些接口与页内字节布局有什么关系？
3. **BlockID** 为什么需要同时保存文件名和块号？如果只保存块号会有什么问题？
4. 如果一次实验中把块大小从 400 改成更大的值，可能影响哪些上层模块或测试？

第 3 章 日志管理实验

3.1 实验目的

本章用于理解 DBMS_C 如何通过 LogManager 组织日志记录，为后续事务提交、回滚和恢复机制提供基础。日志管理层并不直接解释事务语义，也不执行恢复算法；它的职责是把一条条日志记录追加到日志文件中，在需要时刷盘，并提供从后向前读取日志记录的基础能力。

本章实验只覆盖 Log/LogManager.h 和 Log/LogManager.c 中已经存在的真实接口：初始化日志管理器、追加日志记录、flush 日志页、通过 LogIterator 读取日志记录，以及重新打开后读回已经刷盘的日志内容。

3.2 模块作用

LogManager 位于文件层之上、事务恢复层之下。它依赖 FileManager 完成日志文件的块级读写，依赖 Page 表示内存中的日志页，依赖 BlockID 标识当前日志块。上层事务和恢复模块可以把“写一条日志”“确保某个 LSN 已经刷盘”“从后向前扫描日志”交给这一层完成。

本章不展开 tx/recovery/RecoveryManager.c 和具体日志记录类型。恢复算法会在第 5 章事务、并发与恢复实验中再讨论；本章只把日志追加、日志页组织和日志读取基础讲清楚。

3.3 相关源码文件

表 3-1 列出本章直接或间接涉及的真实源码文件。可以看到，日志管理本身代码量不大，但它建立在第 2 章文件页式存储之上。

表 3-1 LogManager 相关源码文件

文件	本章关注点
Log/LogManager.h	声明 LogManager、LogIterator 以及 append、flush、迭代读取接口。
Log/LogManager.c	实现日志页初始化、日志记录追加、LSN 更新、刷盘和逆序读取。
File/FileManager.h	提供日志文件的块级读写、长度查询和追加新块能力。
File/Page.h	提供日志页内 int 和原始字节读写接口。
File/BlockID.h	标识当前日志文件中的具体块号。
Lib/ByteBuffer.h	LogIteratorNext 返回日志 payload 时使用的字节缓冲对象。
Lib/CString.h	表示日志文件名和测试数据库目录名。

阅读时建议先看 LogManagerInit 如何拿到当前日志块，再看 LogManagerAppend 如何在页内写入记录，最后看 LogIteratorNext 为什么能从后写入的记录开始读取。

3.4 核心对象职责

图 3-1 展示了日志管理层的核心对象关系。LogManager 是中心对象，但它并不直接操作磁盘字节，而是通过 FileManager、Page 和 BlockID 协作完成。

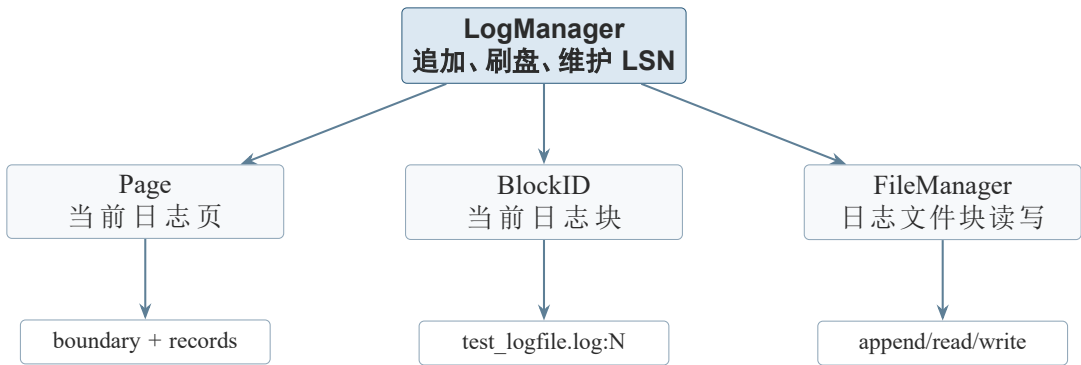


图 3-1 LogManager、FileManager、Page 与 BlockID 的关系

LogManager 维护当前日志文件名、当前日志页、当前日志块、最新 latestLSN 和最近刷盘的 LastSavedLSN。FileManager 负责实际日志文件的块级读写；Page 是日志页在内存中的表示；BlockID 指向日志文件中的一个块。当前实现中存在 LSN：每次 LogManagerAppend 会递增 latestLSN，LogManagerFlush 会把 LastSavedLSN 更新到最新 LSN。

3.5 日志页与日志记录组织

DBMS_C 的日志不是把记录随意散落写入文件，而是通过日志页组织。当前日志页的 0 号偏移处保存一个 boundary，表示本页有效日志记录的起始位置。新日志记录从页尾向前写入：先计算 recordSize，再把位置设为 boundary - recordSize，最后更新页首的 boundary。

图 3-2 描述了这种布局。越新的记录越靠近页前部的 boundary，因此迭代器从 boundary 开始向后移动时，会先读到后写入的记录。

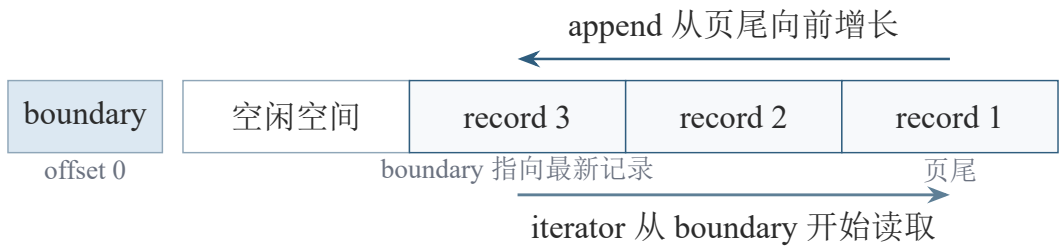


图 3-2 日志页内记录布局示意

每条日志记录由一个内部头部和一段 payload 组成。内部头部在 LogManager.c 中定义为 LogRecordHeader，包含记录总长度和 LSN。payload 的具体含义由上层事务或恢复模块决定，本章测试只把它当作普通字节串。

当当前页空间不足时，LogManagerAppend 会先 flush 当前页，再通过 LogManagerAppendNewBlock 追加一个新的日志块。flush 的作用是把当前日志页

写回日志文件，并记录已经保存到磁盘的 LSN。

3.6 配套测试

本章新增并注册了配套测试文件：

- test/Log/LogManagerBasicTest.c

该测试使用 CMocka，测试名为 LogManagerBasicTest。测试使用独立测试数据库目录和独立日志文件名 test_logfile.log，目录名带进程号和用例计数，避免污染真实数据库文件或重复运行的测试相互影响。

表 3-2 LogManagerBasicTest 覆盖内容

测试函数	验证目标
test_log_manager_append_and_flush	追加一条日志后 LSN 递增，flush 后 LastSavedLSN 更新，日志文件包含一个块。
test_log_manager_iterate_records_reverse_order	连续追加三条文本 payload 后，迭代器按 third、second、first 的顺序读出。
test_log_manager_persists_after_reopen	flush 后销毁并重新打开 FileManager 和 LogManager，仍能读回已经保存的日志。

本章也保留已有 test/Log/LogTest.c，但章节讲解以新增的最小测试为主，因为它只聚焦 LogManager 的基本追加、刷盘和读取能力。

3.7 关键代码讲解

第一段代码来自 Log/LogManager.c 的 LogManagerInit。初始化时，日志管理器会创建一个与文件块大小一致的 Page。如果日志文件还没有块，就追加一个新块；如果已经存在日志块，就读取最后一个日志块作为当前块。

代码 3-1 LogManager 初始化

```
1 LogManager *LogManagerInit(FileManager*fileManager,CString *str){
2     LogManager *logManager = malloc(sizeof (LogManager));
3     logManager->fileManager = fileManager;
4     logManager->logFile = CStringCreateFromCString(str);
5     logManager->logPage = PageInit(logManager->fileManager->blockSize);
6     logManager->LastSavedLSN = 0;
7     logManager->latestLSN = 0;
8     int logSize = FileManagerLength(fileManager,str);
9     if(logSize==0){
10         logManager->currentBlockId = LogManagerAppendNewBlock(logManager);
11     }else{
12         logManager->currentBlockId = BlockIDInit(logManager->logFile,logSize-1);
```

```

13     FileManagerRead(logManager->fileManager, logManager->currentBlockId, logManager->logPage)
14     ;
15     int boundary = PageGetInt(logManager->logPage, 0);
16     if (boundary < logManager->fileManager->blockSize) {
17         LogRecordHeader header;
18         PageGetBytesRaw(logManager->logPage, boundary, (uint8_t*)&header, sizeof(header));
19         logManager->latestLSN = header.lsn;
20     }
21     return logManager;
22 }

```

第二段代码展示了日志追加逻辑。这里的关键点是：日志记录不是从页头顺序写入，而是从页尾向前写入；页首位置 0 保存新的 boundary。

代码 3-2 LogManagerAppend 的核心逻辑

```

1 int LogManagerAppend(LogManager* lm, const uint8_t* payload, uint32_t payloadSize) {
2     uint32_t recordSize = sizeof(LogRecordHeader) + payloadSize;
3     int boundary = PageGetInt(lm->logPage, 0);
4     if (boundary < recordSize + sizeof(int)) {
5         LogManagerFlush(lm);
6         lm->currentBlockId = LogManagerAppendNewBlock(lm);
7         boundary = PageGetInt(lm->logPage, 0);
8     }
9     int pos = boundary - recordSize;
10    LogRecordHeader header = {
11        .length = recordSize,
12        .lsn = ++lm->latestLSN
13    };
14    PageSetBytesRaw(lm->logPage, pos,
15                    (uint8_t*)&header, sizeof(header));
16    PageSetBytesRaw(lm->logPage,
17                    pos + sizeof(header),
18                    payload, payloadSize);
19    PageSetInt(lm->logPage, 0, pos);
20    return header.lsn;
21 }

```

第三段代码展示了 flush。当前实现中，LogManagerFlush 会把当前页写回当前块，并把最近保存 LSN 更新为最新 LSN；LogManagerFlushLSN 则允许上层请求至少刷到某个 LSN。

代码 3-3 日志刷盘

```

1 void LogManagerFlush(LogManager*logManager){
2     FileManagerWrite(logManager->fileManager,logManager->currentBlockId,logManager->logPage);
3     logManager->LastSavedLSN = logManager->latestLSN;
4 }
5
6 void LogManagerFlushLSN(LogManager * logManager,int lsn){
7     if (lsn >= logManager->LastSavedLSN) {
8         FileManagerWrite(logManager->fileManager,logManager->currentBlockId,logManager->logPage
9         );
10        logManager->LastSavedLSN = lsn;
11    }
12 }

```

第四段代码展示了迭代读取。因为 append 把最新记录放在更靠前的位置，迭代器从 boundary 开始向页尾移动时，会自然读出逆序日志。

代码 3-4 LogIterator 读取下一条日志

```

1 ByteBuffer* LogIteratorNext(LogIterator* it) {
2     if (it->currentPos == it->fm->blockSize) {
3         it->blockId->BlockID--;
4         LogIteratorMoveToBlock(it);
5     }
6
7     LogRecordHeader header;
8     PageGetBytesRaw(it->page,
9         it->currentPos,
10        (uint8_t*)&header,
11        sizeof(LogRecordHeader));
12
13    uint32_t payloadSize =
14        header.length - sizeof(LogRecordHeader);
15
16    uint8_t* payload = malloc(payloadSize);
17
18    PageGetBytesRaw(it->page,
19        it->currentPos + sizeof(LogRecordHeader),
20        payload,
21        payloadSize);
22
23    it->currentPos += header.length;
24
25    return bufferInitByteSize(payloadSize, payload);
26 }

```

3.8 运行与观察

在 `D:\textbackslash code\textbackslash DBMS\textbackslash NewDBMS\textbackslash DBMS_C` 下使用固定命令构建和测试：

代码 3-5 构建并运行 Log 相关测试

```
1 cmake -S . -B build
2 mingw32-make -C build -j8 SHELL=cmd.exe
3 ctest --test-dir build --output-on-failure -R Log
```

如果只想运行本章新增测试，可以使用：

代码 3-6 只运行 LogManagerBasicTest

```
1 ctest --test-dir build --output-on-failure -R LogManagerBasicTest
```

图 3-3 概括了追加与刷盘的调用链。上层传入一段 payload，LogManager 加上内部 header 后写入当前日志页；flush 时再把整个页交给 FileManagerWrite 写入日志文件块。

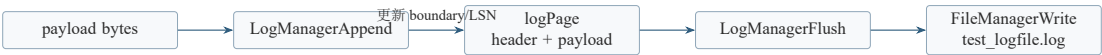


图 3-3 append + flush 调用链

图 3-4 展示了本章测试验证的逆序读取行为。测试依次写入 first、second、third，迭代器读出的顺序是 third、second、first。

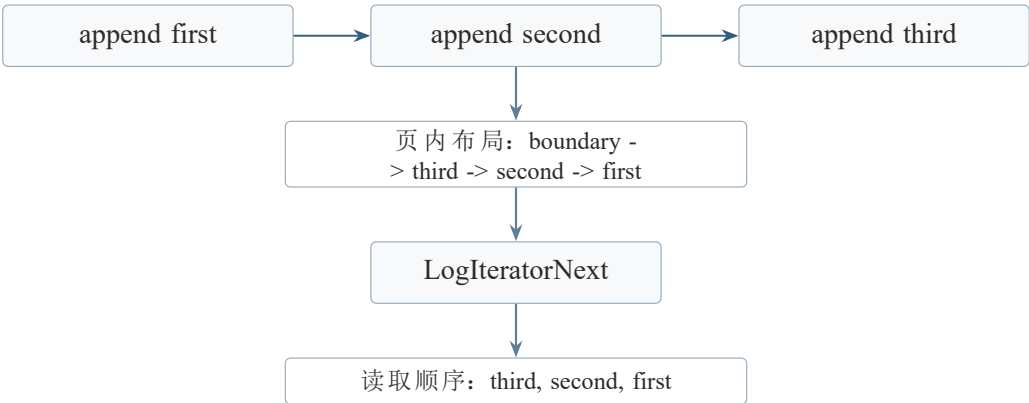


图 3-4 日志逆序读取示意

运行本章实验时，建议重点观察：日志记录是否能被追加；flush 后 LastSavedLSN 是否更新；多条日志记录的读取顺序是否符合当前实现；重新打开后是否能读回已刷盘记录；以及 LogManager 如何依赖第 2 章的 FileManager 和 Page。

3.9 思考题

1. 为什么数据库需要日志，而不是只修改数据页？
2. 为什么日志通常适合顺序追加，而不是频繁在文件中间修改？
3. flush 日志和写回数据页之间可能有什么先后约束？
4. 为什么回滚或恢复过程经常需要从后向前扫描日志？

第 4 章 缓冲区管理实验

4.1 实验目的

本章用于理解 DBMS_C 如何在内存中缓存文件块，减少上层模块直接访问磁盘文件的次数。第 2 章已经看到 `FileManager` 可以把 `Page` 写入文件块或从文件块读回；本章进一步观察 `BufferManager` 如何在文件层之上维护一个固定大小的缓冲池，让上层通过 `pin` 和 `unpin` 使用块。

本章只讨论缓冲区管理的基础行为：缓冲帧、`pin/unpin`、`dirty` 标记、`flush` 写回和替换策略接口。事务回滚、恢复算法和日志记录语义不在本章展开。

4.2 模块作用

`buffer/` 模块位于文件页式存储层之上、记录管理和事务管理层之下。它的核心职责是把磁盘块装入内存 `Buffer`，并在块被修改后按需要写回文件。上层模块不必每次访问记录都直接调用 `FileManagerRead` 或 `FileManagerWrite`，而是先向 `BufferManager` 申请 `pin` 某个 `BlockID`，再通过返回的 `Buffer->page` 读写页内数据。

如果一个 `Buffer` 被 `pin`，表示当前有人正在使用它，不能被替换；如果 `unpin` 后 `pin` 计数归零，它重新变为可用帧。被修改过的 `Buffer` 会通过 `BufferSetModified` 记录事务号和可选 `LSN`，后续 `flush` 时写回底层文件。

4.3 相关源码文件

表 4-1 列出本章涉及的真实源码文件。这里的 `ReplacementPolicy` 是接口，当前默认实现是 `LRUPolicyCreate` 返回的 `LRU` 策略。

从阅读顺序看，可以先看 `Buffer.c` 中单个缓冲帧的状态变化，再看 `BufferManager.c` 如何管理多个 `Buffer`，最后看 `LRU` 策略如何通过统一接口接入。

4.4 核心对象关系

图 4-1 展示了本章对象之间的关系。`BufferManager` 管理一个 `CVector` 形式的缓冲池，池中每个元素是一个 `Buffer*`；每个 `Buffer` 拥有一个 `Page`，并可绑定到某个 `BlockID`。

当前 `BufferManagerInit` 可以接收外部替换策略；如果传入 `NULL`，它会创建默认 `LRU` 策略。测试 `BufferManagerBasicTest` 使用默认 `LRU`，但只验证稳定的缓冲池行为，不把 `LRU` 内部链表细节作为断言目标。

4.5 pin/unpin 机制

`pin` 表示一个块正在被使用。调用 `BufferManagerPin` 时，系统会先查找该块是否已经在缓冲池中；如果已经存在，直接增加该 `Buffer` 的 `pin` 计数；如果不存在，就选择

表 4-1 BufferManager 相关源码文件

文件	本章关注点
buffer/Buffer.h	声明单个缓冲帧，保存 Page、BlockID、pin 计数、事务号和 LSN。
buffer/Buffer.c	实现 BufferPin、BufferUnPin、BufferSetModified、BufferFlush、BufferAssignToBlock。
buffer/BufferManager.h	声明缓冲池、可用帧数量、替换策略和 pin/unpin 等接口。
buffer/BufferManager.c	实现查找已有 Buffer、选择未 pin 帧、pin 块、unpin 帧和按事务刷脏页。
buffer/ReplacementPolicy.h	定义替换策略函数指针接口。
buffer/LRU/LRUCore.h, buffer/LRU/LRUCore.c	实现 LRU 访问记录、victim 查询和移除。
buffer/LRU/LRUPolicy.h, buffer/LRU/LRUPolicy.c	把 LRUCore 包装为 ReplacementPolicy。
File/FileManager.h	Buffer flush 和 assign 时依赖的块级读写接口。
Log/LogManager.h	Buffer flush 时如果存在 LSN，会先调用日志刷盘接口。

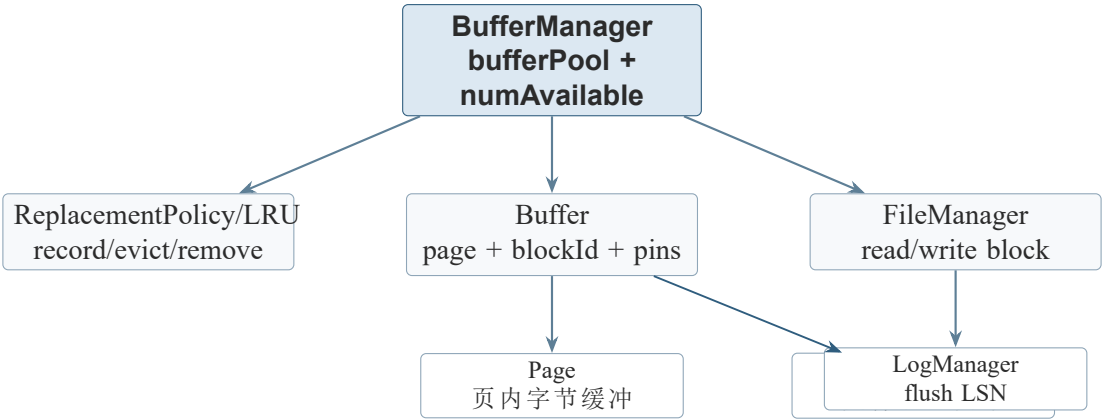


图 4-1 BufferManager、Buffer、Page、BlockID、FileManager 与 LogManager 的关系

一个未 pin 的 Buffer，把它绑定到目标块并读入对应页。调用 `BufferManagerUnpin` 时，pin 计数减少；当 pin 计数降到 0，缓冲帧重新计入可用数量。

图 4-2 展示了单个 Buffer 的状态转换。这里的 dirty 状态不是独立字段，而是由当前实现中的 `txNum >= 0` 表示。

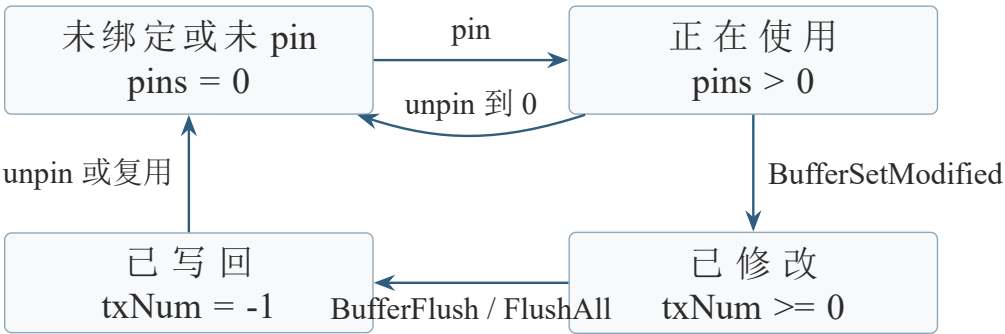


图 4-2 Buffer 状态转换图

图 4-3 展示了 pin/unpin 的调用链。注意，`BufferManagerPin` 会在内部循环调用 `BufferManagerTryToPin`；如果暂时没有可用 Buffer，就等待，超过 `MAX_TIME` 后返回 `NULL`。

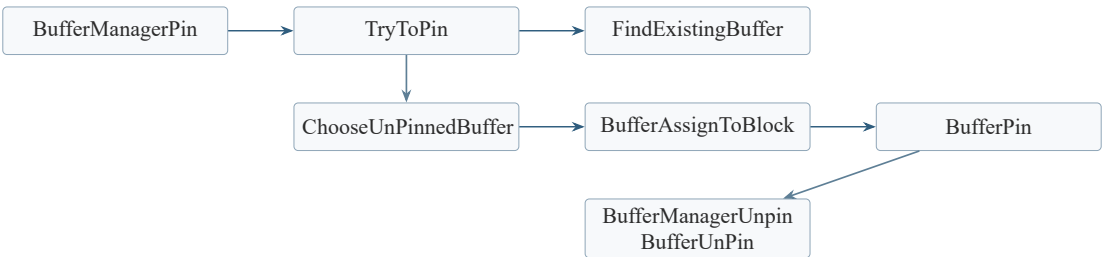


图 4-3 pin/unpin 调用链图

4.6 modified、dirty 与 flush 机制

当前实现中，`BufferSetModified(buffer, txNum, lsn)` 会把 `txNum` 写入 Buffer；如果传入的 `lsn >= 0`，还会记录日志序号。随后 `BufferFlush` 会检查三个条件：Buffer 不为空、已经绑定到某个 `BlockID`、并且 `txNum >= 0`。只有满足这些条件时，Buffer 才会被认为需要写回。

图 4-4 展示了脏页 flush 与文件写回的关系。如果 Buffer 有合法 LSN，flush 会先请求 `LogManagerFlushLSN`；随后再调用 `FileManagerWrite` 把 Page 写回对应文件块。

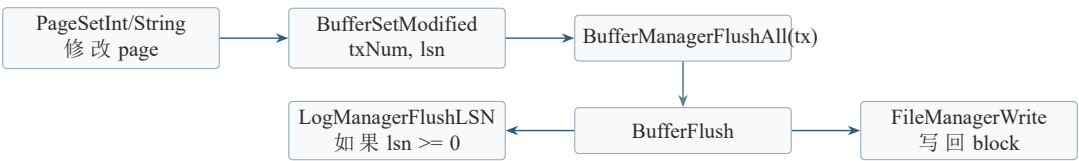


图 4-4 脏页 flush 与文件写回示意图

本章测试使用 `lsn = -1`，只验证 Buffer 修改后通过 `BufferManagerFlushAll` 写回文件；日志先行刷盘的约束会在事务与恢复章节结合真实事务流程再讨论。

4.7 配套测试

本章新增并注册了配套测试文件：

- test/buffer/BufferManagerBasicTest.c

该测试使用独立测试数据库目录，目录名带进程号和用例计数；数据文件为 buffer_basic.tbl,日志文件为 buffer_basic.log。测试使用 FileManager、LogManager 和默认 LRU 策略初始化 BufferManager。

表 4-2 BufferManagerBasicTest 覆盖内容

测试函数	验证目标
test_buffer_manager_pin_and_unpin_updates_available	pin 后可用 Buffer 数减少，unpin 到 0 后恢复。
test_buffer_manager_repin_existing_block	同一 BlockID 重复 pin 返回同一个 Buffer，并正确维护 pin 计数。
test_buffer_manager_dirty_flush_writes_page	修改 Buffer 中的 Page，标记 modified，调用 FlushAll 后能从文件读回。
test_buffer_manager_can_reuse_unpinned_frame	缓冲池满时，unpin 一个帧后可以复用它加载新块。

4.8 关键代码讲解

第一段代码来自 buffer/Buffer.c。它说明单个 Buffer 在绑定新块前会先 flush 自己，随后复制目标 BlockID 并通过 FileManagerRead 读入页面。

代码 4-1 Buffer 绑定到文件块

```
1 void BufferAssignToBlock(Buffer *buffer, BlockID *blockId) {
2     BufferFlush(buffer);
3     buffer->blockId = BlockIDInit(blockId->fileName, blockId->BlockID);
4     FileManagerRead(buffer->fileManager, blockId, buffer->page);
5     buffer->pins = 0;
6 }
```

第二段代码来自 buffer/BufferManager.c 的 BufferManagerTryToPin。这个函数先复用已有缓存；如果块不在池中，就选择未 pin 的 Buffer，并把它分配给目标块。

代码 4-2 BufferManagerTryToPin 的核心逻辑

```

1 Buffer *BufferManagerTryToPin(BufferManager *bm, BlockID *blockId) {
2     Buffer *buffer = BufferManagerFindExistingBuffer(bm, blockId);
3     if (buffer != NULL) {
4         if (buffer->pins == 0)
5             bm->numAvailable--;
6         bm->policy->record_access(bm->policy->impl, buffer->frame_id);
7         BufferPin(buffer);
8         return buffer;
9     }
10
11     buffer = BufferManagerChooseUnPinnedBuffer(bm);
12
13     if (buffer == NULL) {
14         int fid = bm->policy->evict(bm->policy->impl);
15         if (fid < 0)
16             return NULL;
17
18         buffer = *(Buffer**)CVectorAt(bm->bufferPool, fid);
19         if (BufferIsPinned(buffer))
20             return NULL;
21     }
22
23     BufferAssignToBlock(buffer, blockId);
24     bm->numAvailable--;
25     bm->policy->record_access(bm->policy->impl, buffer->frame_id);
26     BufferPin(buffer);
27     return buffer;
28 }

```

第三段代码展示 unpin。只有从 pinned 状态降到 pins == 0 时，numAvailable 才增加；同时替换策略会移除这个 frame 的访问记录。

代码 4-3 BufferManagerUnpin

```

1 void BufferManagerUnpin(BufferManager *bm, Buffer *buffer) {
2     int wasPinned = BufferIsPinned(buffer);
3     BufferUnPin(buffer);
4
5     if (wasPinned && buffer->pins == 0) {
6         bm->numAvailable++;
7         bm->policy->remove(bm->policy->impl, buffer->frame_id);
8     }
9 }

```

第四段代码来自 buffer/Buffer.c 的 BufferFlush。它体现了本章所说的

dirty/flush 规则：只有已绑定块且 `txNum >= 0` 的 Buffer 才会真正写回。

代码 4-4 BufferFlush

```

1 void BufferFlush(Buffer *buffer) {
2     if (buffer == NULL)
3         return;
4
5     if (buffer->blockId == NULL)
6         return;
7     if (buffer->txNum < 0)
8         return;
9
10    if (buffer->lsn >= 0) {
11        LogManagerFlushLSN(buffer->logManager, buffer->lsn);
12    }
13
14    FileManagerWrite(buffer->fileManager, buffer->blockId, buffer->page);
15    buffer->txNum = -1;
16 }

```

4.9 运行与观察

在 `D:\textbackslash code\textbackslash DBMS\textbackslash NewDBMS\textbackslash DBMS_C` 下使用固定命令构建和测试：

代码 4-5 构建并运行 Buffer 相关测试

```

1 cmake -S . -B build
2 mingw32-make -C build -j8 SHELL=cmd.exe
3 ctest --test-dir build --output-on-failure -R Buffer

```

如果只想运行本章新增测试，可以使用：

代码 4-6 只运行 BufferManagerBasicTest

```

1 ctest --test-dir build --output-on-failure -R BufferManagerBasicTest

```

观察测试输出时，可以重点看四件事：第一，`pin` 后 `numAvailable` 是否减少；第二，同一块重复 `pin` 是否仍然返回同一个 Buffer；第三，`unpin` 到 0 后 Buffer 是否重新变为可用；第四，修改页并调用 `BufferManagerFlushAll` 后，底层文件能否读回写入值。

4.10 思考题

1. 为什么数据库需要缓冲池，而不是每次访问记录都直接读写文件？

2. `pin` 计数为什么不能简单设计成布尔值？
3. 当前实现中 `dirty` 状态由 `txNum >= 0` 表示，这种设计有什么优点和局限？
4. `flush` 脏页前为什么可能需要先 `flush` 日志？

第 5 章 事务、并发与恢复实验

5.1 实验目的

本章用于理解 DBMS_C 中事务层的基本职责。事务并不是单独完成读写的模块，它会把文件层、日志层、缓冲区层、并发控制层和恢复层组织在一起，对上层提供相对统一的读写入口。

本章当前已经完成三个基础阶段：5A 验证 Transaction 如何通过 BufferList 使用 BufferManager，并在提交时把修改写回到底层文件；5B 验证 LockTable 和 ConCurrencyManager 的基础加锁与释放行为；5C 验证 RecoveryManager 通过旧值日志完成最小整数回滚。死锁检测和崩溃恢复属于后续扩展内容，本章只保留边界说明，不提前展开。

5.2 模块作用

tx/ 模块位于查询执行、记录管理和底层缓冲区管理之间。上层模块不需要直接操作每一个 Buffer，而是可以通过事务接口 pin 块、读取页内数据、写入页内数据、提交或回滚事务。

在当前实现中，Transaction 初始化时会创建三个重要的协作对象：RecoveryManager、ConCurrencyManager 和 BufferList。其中 BufferList 记录当前事务已经 pin 的块和对应 Buffer，使事务可以在后续读写时找到已经固定的缓冲页。

5.3 相关源码文件

表 5-1 列出本轮事务基础实验涉及的真实源码文件。锁和恢复相关文件虽然会被 Transaction 调用，但本轮只说明它们在事务对象中的位置，不测试其内部细节。

从阅读顺序看，可以先阅读 Transaction.h，了解事务暴露给上层的接口；再看 Transaction.c 中这些接口如何转发到 BufferList、BufferManager、ConCurrencyManager 和 RecoveryManager；最后再回到第 4 章的缓冲区管理内容，理解提交时为什么要释放和写回 Buffer。

5.4 Transaction 在系统中的位置

图 5-1 展示了当前事务对象与周边模块的关系。事务层向上承接记录、扫描和更新执行，向下连接缓冲区、文件和日志。并发控制和恢复管理是事务对象内部的重要协作者，但本轮测试只验证基础读写与提交路径。

这张图中的虚线表示恢复管理在提交时会要求缓冲区刷出某个事务修改过的页。由于本轮测试使用 TransactionSetInt(..., false)，它不生成数据修改日志，只验证事务提交后 Buffer 能写回文件；日志语义将在恢复实验中再单独观察。

表 5-1 Transaction 基础实验相关源码文件

文件	本章关注点
tx/Transaction.h	声明事务状态、事务结构体和 pin、get、set、commit 等接口。
tx/Transaction.c	实现事务初始化、读写、提交、回滚以及对 BufferList 的调用。
tx/BufferList.h	声明事务私有的块到 Buffer 映射和 pin 列表。
tx/BufferList.c	实现事务 pin/unpin 块、查找 Buffer、提交时释放所有 Buffer。
buffer/BufferManager.h	提供缓冲区池、pin/unpin、flush 等能力。
File/FileManager.h	提供文件块读写能力，是提交后验证持久化结果的基础。
Log/LogManager.h	为 RecoveryManager 写入事务日志提供底层日志管理能力。
tx/concurrency/ConcurrencyManager.h	被 Transaction 调用以申请读写锁；本轮不展开锁实现。
tx/recovery/RecoveryManager.h	被 Transaction 调用以记录和提交日志；本轮不展开恢复算法。

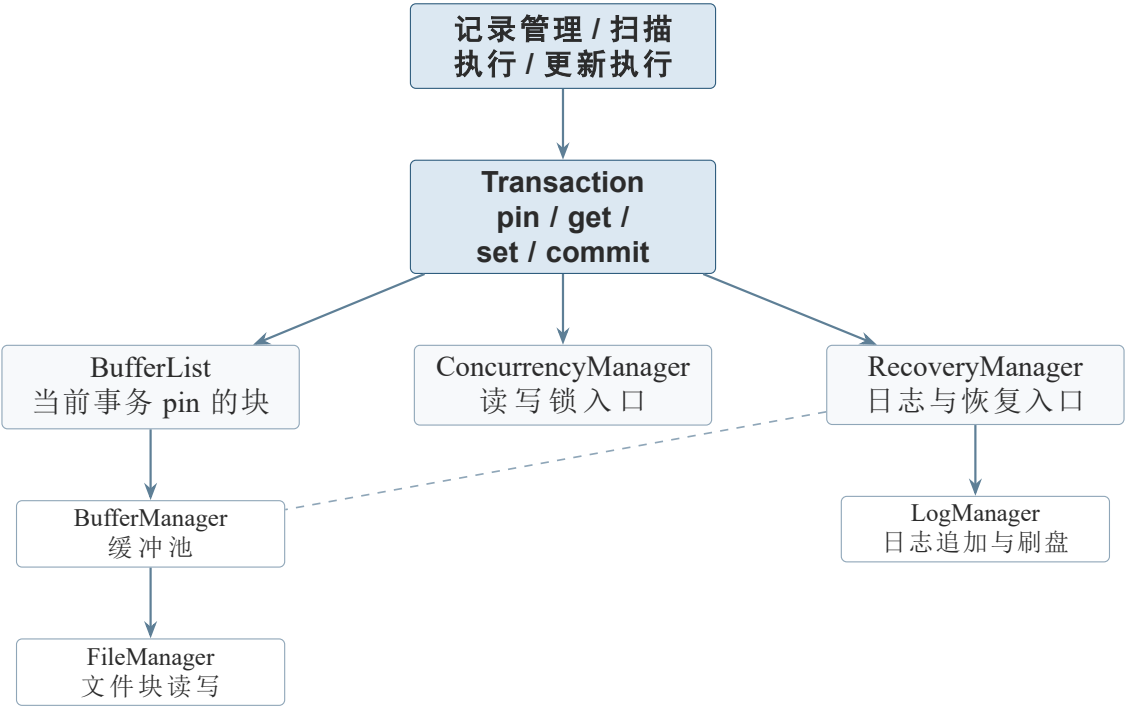


图 5-1 Transaction 与缓冲区、并发控制和恢复管理的关系

5.5 BufferList 的作用

BufferList 是事务层和缓冲区层之间的一层薄封装。它做两件事：第一，把当前事务 pin 过的 BlockID 映射到对应 Buffer；第二，维护一个 pin 列表，方便事务提交或回滚时一次性释放该事务固定过的所有 Buffer。

图 5-2 展示了这个关系。事务对同一个块执行读写时，并不是每次都重新从 BufferManager 查找，而是可以通过 BufferListGetBuffer 找到当前事务已经 pin 的 Buffer。

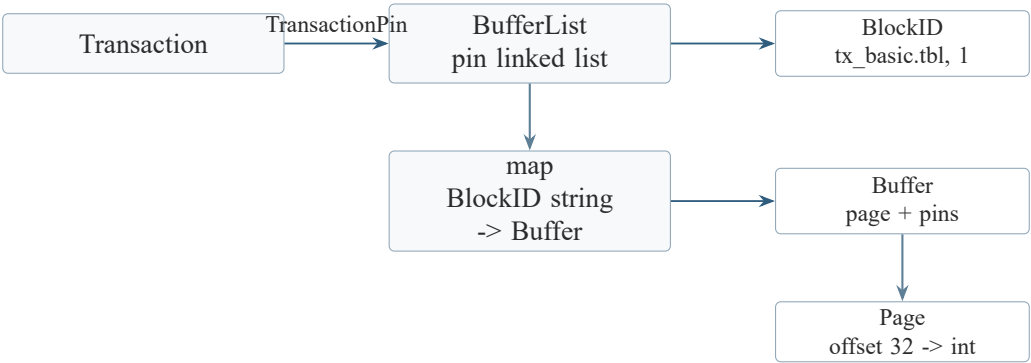


图 5-2 BufferList 维护事务使用过的 BlockID 和 Buffer

5.6 事务基础读写与提交流程

本轮实验只观察一条最小事务链路：初始化事务，pin 一个文件块，在该块对应的 Page 中写入整数，提交事务，再通过 FileManagerRead 从文件中读回验证结果。图 5-3 给出了这个调用链。

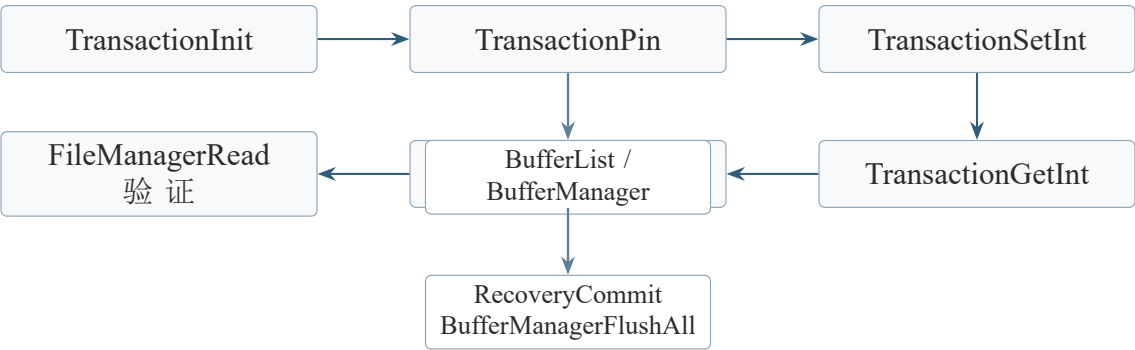


图 5-3 Transaction 基础读写与提交调用链

这条链路刻意避开了回滚恢复断言,因为回滚需要结合 RecoveryManager 的日志记录 and 旧值恢复语义来设计测试。为了保持章节边界清晰，本轮只确认事务与 BufferList、BufferManager 的基础协作。

5.7 配套测试

本章新增并注册了配套测试文件：

- test/tx/TransactionBasicTest.c

该测试使用独立数据库目录，目录名包含进程号和用例计数，数据文件为 tx_basic.tbl，日志文件为 tx_basic.log。测试完成后会尽量清理测试目录，避免污染真实数据库文件。

表 5-2 TransactionBasicTest 覆盖内容

测试函数	验证目标
test_transaction_init_reports_environment	创建 Transaction 后，事务处于 RUN 状态，能报告块大小、可用 Buffer 数，并持有 BufferList、Concurrency-Manager、RecoveryManager。
test_transaction_pin_and_unpin_updates_available_buffers	通过 Transaction pin 一个 BlockID 后，可用 Buffer 数减少；unpin 后可用 Buffer 数恢复。
test_transaction_set_int_commit_persists_value	通过 TransactionSetInt 写入整数，提交后使用 FileManagerRead 从文件块读回同一整数。

运行本章测试可以使用：

代码 5-1 运行事务基础测试

```
1 ctest --test-dir build --output-on-failure -R Transaction
```

完整回归测试仍然使用：

代码 5-2 完整 CTest 回归

```
1 ctest --test-dir build --output-on-failure
```

5.8 关键代码讲解

第一段代码来自 tx/Transaction.c 的 TransactionInit。事务初始化时会保存文件管理器和缓冲区管理器，并创建恢复管理器、并发管理器和 BufferList。

代码 5-3 TransactionInit 初始化事务协作对象

```

1 Transaction* TransactionInit(FileManager*fileManager,
2                             LogManager*logManager,
3                             BufferManager*bufferManager){
4     Transaction *transaction = malloc(sizeof (Transaction));
5     transaction->txNum = TransactionNextTxNumber(transaction);
6     transaction->fileManager = fileManager;
7     transaction->bufferManager = bufferManager;
8     transaction->recoveryManager =
9         RecoveryManagerInit(transaction, transaction->txNum,
10                             LogManager, bufferManager);
11     transaction->concurrencyManager = ConcurrencyManagerInit();
12     transaction->bufferList = BufferListInit(bufferManager);
13     transaction->code = TX_TRANSACTION_RUN;
14     return transaction;
15 }

```

第二段代码来自 tx/BufferList.c 的 BufferListPin。它向 BufferManager 申请目标块对应的 Buffer，并把块到 Buffer 的映射记录到当前事务的 BufferList 中。

代码 5-4 BufferListPin 记录事务 pin 的块

```

1 void BufferListPin(BufferList *bufferList, BlockID *blockId){
2     Buffer *buffer = BufferManagerPin(bufferList->bufferManager, blockId);
3     BufferTEMP *bufferTemp = BufferTEMPInit();
4     bufferTemp->buffer = buffer;
5     map_set(bufferList->buffers,
6             CStringGetPtr(BlockID2CString(blockId)),
7             *bufferTemp);
8
9     BlockIDNode *blockIdNode = bufferList->pin;
10    while(blockIdNode->next != NULL){
11        blockIdNode = blockIdNode->next;
12    }
13    BlockIDNode *NewBlockIDNode = malloc(sizeof(BlockIDNode));
14    NewBlockIDNode->blockId = BlockIDInit(blockId->fileName,
15                                          blockId->BlockID);
16    NewBlockIDNode->next = NULL;
17    blockIdNode->next = NewBlockIDNode;
18 }

```

第三段代码来自 tx/Transaction.c 的 TransactionSetInt。写入前事务会申请写锁，随后从 BufferList 找到 Buffer，把整数写入 Page，并把 Buffer 标记为本事务修改过。

代码 5-5 TransactionSetInt 写入页内整数

```

1 void TransactionSetInt(Transaction*transaction, BlockID *blockId,
2     int offset, int val, bool okToLog){
3     ConCurrencyManagerXLock(transaction->conCurrencyManager, blockId);
4     Buffer *buffer = BufferListGetBuffer(transaction->bufferList, blockId);
5     int lsn = -1;
6     if(okToLog){
7         lsn = RecoverySetInt(transaction->recoveryManager,
8             buffer, offset, val);
9     }
10    Page *page = buffer->page;
11    PageSetInt(page, offset, val);
12    BufferSetModified(buffer, transaction->txNum, lsn);
13 }

```

第四段代码来自 tx/Transaction.c 的 TransactionCommit。提交时先走恢复管理器的提交入口，再释放锁和当前事务 pin 过的 Buffer，最后把事务状态改为已提交。

代码 5-6 TransactionCommit 提交事务

```

1 void TransactionCommit(Transaction*transaction){
2     TransactionTraceCommit(transaction);
3     RecoveryCommit(transaction->recoveryManager);
4     printf("transaction %d commit\n", transaction->txNum);
5     ConCurrencyManagerRelease(transaction->conCurrencyManager);
6     BufferListUnpinAll(transaction->bufferList);
7     transaction->code = TX_TRANSACTION_COMMIT;
8 }

```

5.9 5B 并发锁基础实验

事务层在读写块之前会先进入并发控制模块。当前实现中，读操作通过 ConCurrencyManagerSLock 获取共享锁，写操作通过 ConCurrencyManagerXLock 获取排他锁；提交或回滚时，ConCurrencyManagerRelease 会释放当前事务管理器记录的所有锁。

LockTable 是全局锁表，内部用 map_int_t 记录每个 BlockID 对应的锁状态。当前约定是：值为正数表示共享锁数量，值为 -1 表示排他锁，值不存在或查询为 0 表示没有锁。ConCurrencyManager 则是事务侧的锁管理器，它用 map_str_t 记录当前事务已经持有的锁类型，例如 "S" 或 "X"。

图 5-4 展示了事务、并发管理器、锁表和块标识之间的关系。

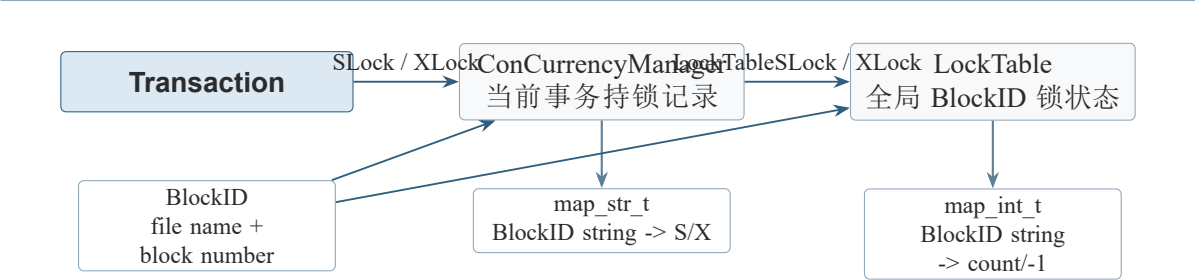


图 5-4 Transaction、ConCurrencyManager、LockTable 与 BlockID 的关系

表 5-3 给出本章采用的锁语义。当前测试不构造冲突等待，只验证单个锁表或单个事务管理器在非冲突路径上的状态变化。

表 5-3 S 锁与 X 锁的基础兼容关系

已有锁	新申请 S 锁	新申请 X 锁
无锁	可以获取，共享计数增加	可以获取，锁表值变为 -1
S 锁	理论上可共享；本轮不测试多事务共享计数	需要等待其他 S 锁释放；本轮不测试阻塞升级
X 锁	需要等待 X 锁释放；本轮不测试阻塞等待	同一管理器重复获取会直接返回

读写操作进入并发控制的调用链如图 5-5 所示。读路径调用共享锁，写路径调用排他锁，最终都以 BlockID 的字符串形式作为锁表键。

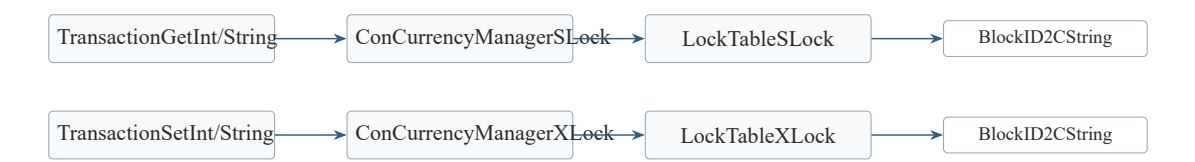


图 5-5 读写操作获取 S 锁和 X 锁的调用链

锁释放由 ConCurrencyManagerRelease 统一完成。它遍历当前事务管理器的本地锁记录，对每个键调用 LockTableUnLock，再从本地 map 中删除该键。图 5-6 展示了这个过程。

本轮新增的并发基础测试文件为：

- test/tx/ConcurrencyBasicTest.c

单独运行 5B 测试可以使用：

代码 5-7 运行并发锁基础测试

```
1 ctest --test-dir build --output-on-failure -R Concurrency
```

本轮没有测试复杂阻塞场景，原因有三点：第一，当前 LockTableSLock 和 LockTableXLock 在冲突时采用等待循环，并通过时间判断退出，测试会变慢且不稳定；

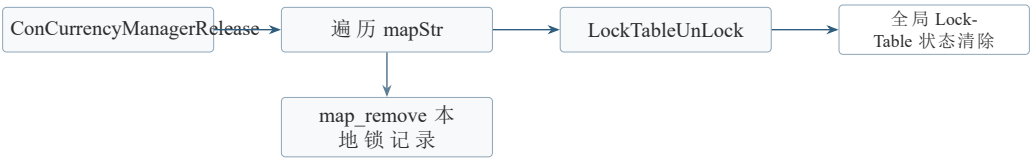


图 5-6 release all 锁释放示意图

表 5-4 ConcurrencyBasicTest 覆盖内容

测试函数	验证目标
test_lock_table_slock_then_unlock	直接验证 LockTable 获取 S 锁后锁值为 1，释放后锁值恢复为 0。
test_lock_table_xlock_then_unlock	直接验证 LockTable 获取 X 锁后锁值为 -1，释放后锁值恢复为 0。
test_concurrency_manager_tracks_s_and_x_locks	验证 ConCurrencyManager 会在本地记录 S/X 锁类型，并能识别 X 锁。
test_concurrency_manager_release_allows_relock	验证 release all 后，本地锁记录清空，并且另一个管理器可以重新获取同一 BlockID 的锁。

第二，本章目标是锁基础状态，不是多事务调度压力测试；第三，DeadlockDetector 的等待图和环检测属于后续独立实验，不能在没有专门测试设计的情况下混入本轮。

5.10 5C 恢复与回滚基础实验

恢复管理模块负责在事务修改数据前记录足够的信息，使事务回滚时可以把被修改的数据恢复到旧值。当前实现中，TransactionSetInt 和 TransactionSetString 在 okToLog 为 true 时，会调用 RecoverySetInt 或 RecoverySetString 记录修改前的旧值；随后才真正修改缓冲页。

RecoveryManager 位于事务层内部，连接 LogManager 与 BufferManager。日志记录的具体格式由 LogRecord 负责解析和执行 undo。图 5-7 展示了当前 rollback 基础路径中几个对象之间的关系。

为什么要在修改前记录旧值？因为 rollback 的核心不是“撤销函数调用”，而是把数据页恢复成修改之前的内容。以整数写入为例，RecoverySetInt 会先从当前 Buffer->page 读出旧值，再调用 SetIntRecordWriteToLog 把事务号、文件块、偏移量和旧值写入日志。图 5-8 展示了这个记录过程。

rollback 时，RecoveryDoRollback 会通过日志迭代器从后向前读取日志，找到当前事务的日志记录。如果遇到 SETINT 记录，就调用该记录的 undo 函数，把记录中的旧值写回原来的 BlockID 和 offset；如果遇到当前事务的 START 记录，说明这个事务的回滚扫描结束。图 5-9 展示了这条基础链路。

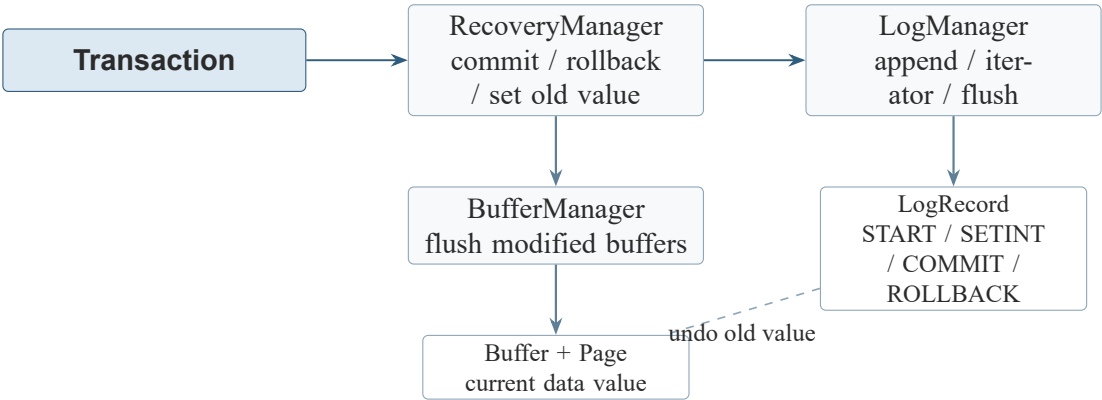


图 5-7 Transaction、RecoveryManager、LogManager 与 BufferManager 的关系

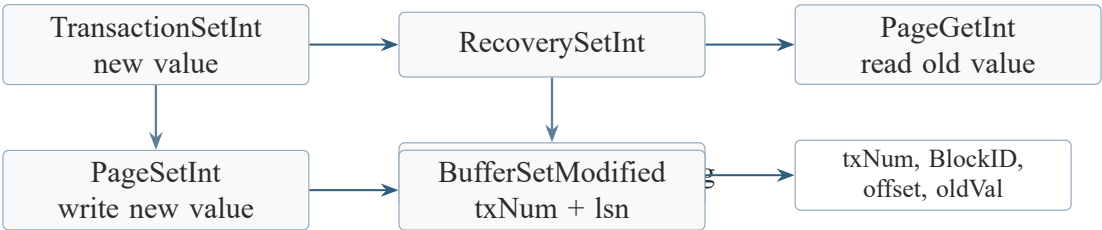


图 5-8 SETINT 修改前记录旧值日志的过程

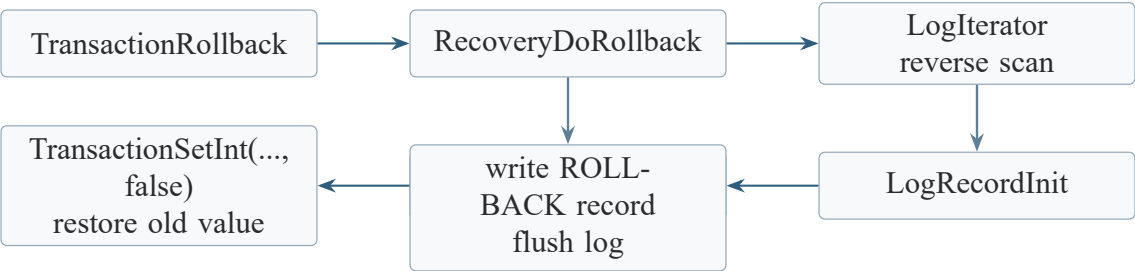


图 5-9 rollback 恢复旧值的基础流程

表 5-5 对比了本章已经测试到的提交和回滚行为。这里的回滚只覆盖单事务、单整数旧值恢复，不代表完整崩溃恢复能力。

表 5-5 commit 与 rollback 基础行为对比

操作	当前实现中的核心动作	本章测试覆盖情况
commit	flush 当前事务修改过的 Buffer，写入 COMMIT 日志，释放锁和 Buffer。	TransactionBasicTest 验证提交后 int 能从文件读回。
rollback	倒序扫描日志，对 SETINT/SETSTRING 调用 undo，写入 ROLLBACK 日志，释放锁和 Buffer。	RecoveryBasicTest 验证 SETINT 旧值能恢复。
crash recovery	扫描日志，跳过已提交或已回滚事务，对未完成事务执行 undo。	本章不测试，留作后续扩展。

本轮新增的恢复基础测试文件为：

- test/tx/RecoveryBasicTest.c

该测试使用独立数据库目录、独立数据文件和日志文件。测试流程是：先启动一个事务写入初始整数并提交；再启动第二个事务读取旧值、使用 TransactionSetInt(..., true) 修改该整数；随后调用 TransactionRollback；最后从文件块读回并断言值恢复为旧值。

单独运行 5C 测试可以使用：

代码 5-8 运行恢复与回滚基础测试

```
1 ctest --test-dir build --output-on-failure -R Recovery
```

本轮没有测试字符串回滚、多事务交错和崩溃恢复。字符串日志路径在当前实现中涉及字符串生命周期和记录长度细节，适合单独补测试；多事务交错会同时牵涉锁、日志顺序和缓冲区替换；崩溃恢复需要重新打开数据库环境并验证未完成事务的 undo 结果，应作为后续更完整的恢复实验推进。

5.11 运行与观察

- 本章的建议观察顺序如下：
1. 运行 ctest --test-dir build --output-on-failure -R Transaction，确认事务基础测试通过。
 2. 运行 ctest --test-dir build --output-on-failure -R Concurrency，确认并发锁基础测试通过。
 3. 运行 ctest --test-dir build --output-on-failure -R Recovery，确认恢复与回滚基础测试通过。

4. 观察 TransactionInit 是否为事务创建必要的协作对象。
5. 观察 pin/unpin 前后 TransactionAvailableBufs 的变化，理解事务如何占用和释放 Buffer。
6. 观察 TransactionSetInt 写入后，提交是否能让 FileManagerRead 从文件块中读回相同整数。
7. 观察 LockTable 中 S 锁、X 锁的数值表示，以及 ConCurrencyManagerRelease 如何清空本地锁记录。
8. 观察 TransactionRollback 是否能借助 SETINT 旧值日志恢复修改前的整数。
9. 注意本章当前没有断言 DeadlockDetector 和崩溃恢复，这些内容必须等对应测试补齐后再写入实验正文。

5.12 后续小节 TODO

表 5-6 记录第 5 章后续实验边界。当前只完成 5A 事务基础，后续内容仍按“先补 CTest，再写章节”的规则推进。

表 5-6 第 5 章后续实验安排

阶段	内容	当前状态
5A	Transaction、BufferList、BufferManager 的基础协作	已新增 TransactionBasicTest 并通过。
5B	LockTable 与 ConCurrencyManager 基础加锁释放实验	已新增 ConcurrencyBasicTest 并通过；DeadlockDetector 暂不展开。
5C	RecoveryManager、LogRecord 与整数旧值回滚实验	已新增 RecoveryBasicTest 并通过；崩溃恢复暂不展开。

5.13 思考题

1. 为什么事务层不直接读写文件，而是通过 BufferManager 和 BufferList 间接访问 Page？
2. Transaction 为什么需要同时持有 ConcurrencyManager 和 RecoveryManager？
3. 如果事务提交时没有释放已经 pin 的 Buffer，后续模块可能遇到什么问题？
4. 为什么回滚测试不能只检查 TransactionRollback 是否被调用，而要结合日志和旧值恢复结果来验证？

第 6 章 记录管理实验

6.1 实验目的

本章用于理解 DBMS_C 如何把上层的“表、字段、记录”落实为页内的固定长度记录槽。前面章节已经看到，`FileManager` 负责文件块读写，`BufferManager` 负责把块缓存到内存，`Transaction` 负责通过缓冲区读写页内数据；记录管理层则站在这些能力之上，进一步提供表结构描述、字段偏移计算、记录标识和顺序扫描能力。

本章只讨论 `record/` 模块本身，不提前展开元数据目录、SQL 解析、查询计划和优化器内容。通过本章实验，读者应能回答三个问题：表结构如何变成页内布局，记录如何通过 slot 存放在 Page 中，`TableScan` 如何在表文件中插入并顺序读出记录。

6.2 模块作用

`record/` 模块位于事务和缓冲区之上、元数据和查询执行之下。它不直接理解 SQL 语句，也不维护系统 catalog，而是提供更底层的记录组织能力：`Schema` 描述字段集合，`Layout` 计算每个字段在记录槽中的偏移，`RID` 标识记录位置，`RecordPage` 在单个页内管理 slot，`TableScan` 在表文件的一个或多个块之间顺序移动。

本章新增的 `RecordBasicTest` 选择了最小闭环：先独立验证 `Schema`、`Layout` 和 `RID`，再通过 `Transaction`、`RecordPage` 和 `TableScan` 写入一条记录并读回。这样可以确认记录管理层的关键抽象已经能独立工作，同时避免把第 7 章以后的元数据和 SQL 执行链路混入本章。

6.3 相关源码文件

表 6-1 列出本章直接展开的真实源码文件。阅读时建议从 `Schema.h` 和 `Layout.h` 开始，因为它们决定一条记录在页中的形状；再看 `RecordPage.c`，理解 slot 如何被格式化、插入、删除和查找；最后看 `TableScan.c`，观察它如何把单页记录管理扩展为表文件扫描。

这些文件共同形成记录管理层的最小工作面。暂不展开的后续模块包括 `metadata/`、`parse/`、`plan/` 和 `query/` 中的查询组合逻辑，它们会在后续章节基于新的 CTest 单独说明。

6.4 核心对象职责

图 6-1 展示了本章几个核心对象的关系。`Schema` 和 `Layout` 负责“记录长什么样”，`RID` 负责“记录在哪里”，`RecordPage` 负责“页内怎么放”，`TableScan` 负责“表文件中怎么顺序移动”。

在当前实现中，`Layout` 生成的每条记录槽都从一个 int 标志位开始，后面依次排列字段。`RecordPage` 通过这个标志位区分空槽和已使用槽，`TableScan` 则把 `RecordPage` 封装起来，使上层可以按“当前记录”的方式读写字段。

表 6-1 Record 模块相关源码文件

文件	本章关注点
record/Schema.h, record/Schema.c	定义字段集合，支持添加 int/string 字段、查询字段类型和长度。
record/Layout.h, record/Layout.c	根据 Schema 计算字段 offset 和每个记录槽的 SlotSize。
record/RID.h, record/RID.c	用 block number 和 slot 标识一条记录的位置。
record/RecordPage.h, record/RecordPage.c	在一个页中格式化 slot、查找空 slot、插入记录、读写字段值。
record/TableScan.h, record/TableScan.c	在表文件上顺序扫描、插入、删除和读写当前记录。
tx/Transaction.h	RecordPage 和 TableScan 通过事务接口读写块内 Page。
File/BlockID.h	标识记录所在的文件块。

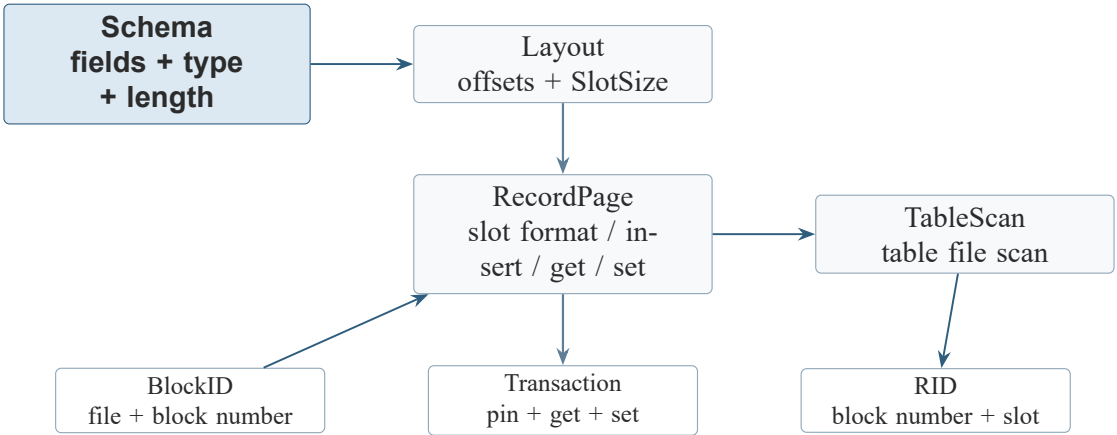
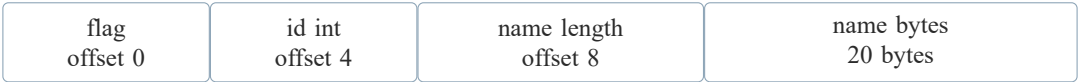


图 6-1 Schema、Layout、RecordPage、TableScan 与 RID 的关系

6.5 Schema 与 Layout

Schema 保存字段链表和字段信息映射。调用 SchemaAddIntField 会添加类型为 FILE_INFO_CODE_INTEGER、长度为 4 的字段；调用 SchemaAddStringField 会添加类型为 FILE_INFO_CODE_VARCHAR 的字符串字段，并保存传入的最大长度。

Layout 根据 Schema 计算页内记录布局。当前实现把每个 slot 的第一个 sizeof(int) 字节留给状态标志，字段从这个位置之后开始排列。int 字段占用 sizeof(int) 字节，字符串字段占用“长度前缀 int + 字符数据长度”。图 6-2 用本章测试中的 id int, name varchar(20) 展示这一布局。



SlotSize = sizeof(int) + sizeof(int) + sizeof(int) + 20

图 6-2 id/name 记录槽的页内字节布局示意

6.6 RID 与记录位置

RID 是记录标识符，只保存两个整数：BlockNum 和 Slot。它不保存表名，因为 TableScan 已经知道自己正在扫描哪个表文件；当需要定位表内某条记录时，block number 指向文件中的块号，slot 指向该块中的记录槽号。

本章测试只验证 RID 的最小行为：初始化后能保存 block 和 slot，两个 RID 能比较是否相等，RIDToString 能生成类似 (3,7) 的字符串形式。更复杂的 RID 定位路径与 TableScanMoveToRid 有关，本轮不把它作为测试目标。

6.7 RecordPage 的 slot 生命周期

RecordPage 管理单个 Page 中的多个 slot。调用 RecordPageFormat 会逐个 slot 写入 RECORD_PAGE_EMPTY 标志，并把每个字段初始化为 0 或空字符串。随后 RecordPageInsertAfter 会查找下一个空 slot，将其标志改为 RECORD_PAGE_USED，再由 RecordSetInt 和 RecordSetString 写入字段值。

图 6-3 展示了一个 slot 的基本生命周期。本章测试覆盖 format、insert、set、get 和 next 查找，不覆盖复杂删除重用场景，因为已有旧测试 RecordTest 中已经包含 slot 管理相关断言。

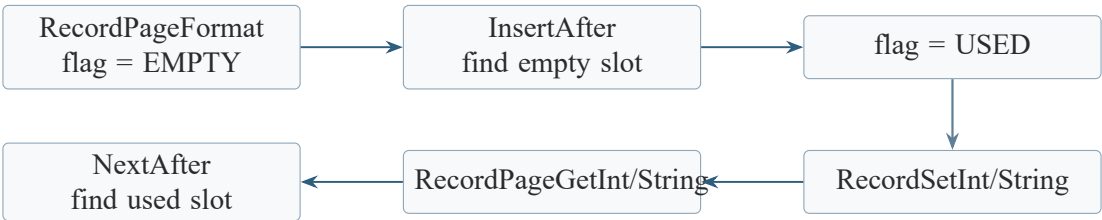


图 6-3 RecordPage 中一个记录槽的基本生命周期

6.8 TableScan 顺序插入与读回

TableScan 把单页的 RecordPage 操作扩展到整个表文件。初始化时，它会把表名拼接成 .tbl 文件名；如果文件长度为 0，就追加新块并格式化第一页；否则移动到第 0 块。插入时，TableScan 在当前 RecordPage 中查找空 slot；如果当前块已满，就移动或追加下一个块。

当前 Scan 体系中，TableScan 的公开读写函数内部会把传入参数转换为 Scan*，再访问 scanUnion.tableScan。因此本章测试不会直接把裸 TableScan* 传给这些函数，而是先调用 ScanInit(tableScan, SCAN_TABLE_CODE) 进行包装。图 6-4 展示本章测试采用的最小链路。

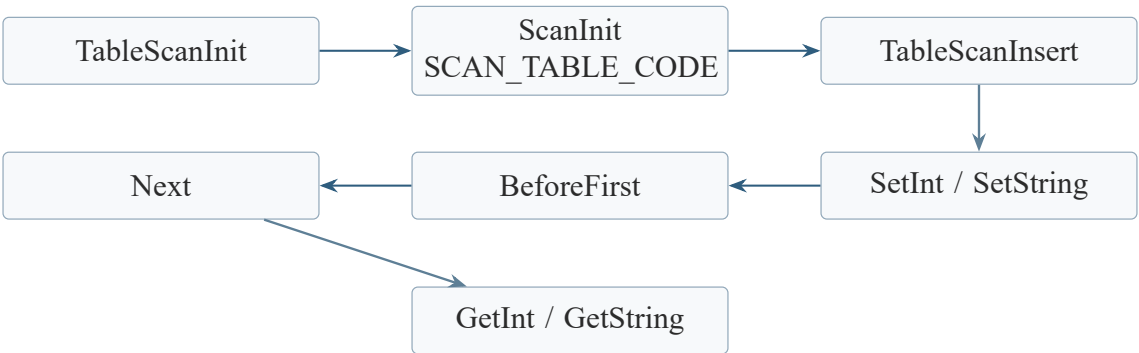


图 6-4 TableScan 插入并顺序读回记录的调用链

6.9 配套测试

本章新增并注册了配套测试文件：

- test/record/RecordBasicTest.c

该测试使用独立数据库目录，目录名包含进程号和用例计数；数据文件包括 record_basic.tbl 和 record_basic_scan.tbl，日志文件为 record_basic.log。测试结束时尽量删除这些文件和目录，避免污染真实数据库数据。

单独运行本章测试可以使用：

代码 6-1 运行 Record 相关测试

```
1 ctest --test-dir build --output-on-failure -R Record
```

只运行本章新增测试可以使用：

代码 6-2 只运行 RecordBasicTest

```
1 ctest --test-dir build --output-on-failure -R RecordBasicTest
```

表 6-2 RecordBasicTest 覆盖内容

测试函数	验证目标
test_schema_adds_int_and_string_fields	添加 int/string 字段后，字段存在，类型和长度正确。
test_layout_computes_offsets_and_slot_size	Layout 为 id/name 字段计算正确 offset，并得到符合当前实现的 SlotSize。
test_rid_stores_identity	RID 保存 block/slot，能比较相等性，并能生成字符串表示。
test_record_page_insert_set_and_read_fields	RecordPage 格式化页面后插入 slot，写入 id/name，再读回同样的值。
test_table_scan_insert_and_read_record	TableScan 经 Scan 包装后插入一条记录，回到表头顺序读出该记录。

6.10 关键代码讲解

第一段代码来自 record/Layout.c 的 LayoutInit。它从 slot 标志位之后开始放置字段，并把每个字段名映射到当前位置。

代码 6-3 LayoutInit 计算字段偏移

```
1 int pos = sizeof(int);
2 FieldNode *fieldNode = schema->fields;
3 while(fieldNode != NULL){
4     map_set(layout->offsets, CStringGetPtr(fieldNode->fileName), pos);
5     pos += LayoutLengthInBytes(schema, fieldNode->fileName);
6     fieldNode = fieldNode->next;
7 }
8 layout->SlotSize = pos;
```

第二段代码来自 record/RecordPage.c 的 RecordPageFormat。格式化页面时，每个 slot 都先写入空标志，再按字段类型写入默认值。

代码 6-4 RecordPageFormat 初始化页内 slot

```
1 while (RecordPageIsValidSlot(recordPage, slot)){
2     TransactionSetInt(recordPage->transaction, recordPage->blockId,
3         RecordPageOffset(recordPage, slot),
4         RECORD_PAGE_EMPTY, false);
5     Schema *schema = recordPage->layout->schema;
6     FieldNode *fieldNode = schema->fields;
7     while (fieldNode != NULL){
```

```

8      CString *fldName = fieldNode->fileName;
9      int fldPos = RecordPageOffset(recordPage, slot)
10         + LayoutOffset(recordPage->layout, fldName);
11      if(SchemaType(schema, fldName) == FILE_INFO_CODE_INTEGER){
12          TransactionSetInt(recordPage->transaction,
13              recordPage->blockId, fldPos, 0, false);
14      }else{
15          TransactionSetString(recordPage->transaction,
16              recordPage->blockId, fldPos,
17              CStringCreateFromCStr(""), false);
18      }
19      fieldNode = fieldNode->next;
20  }
21  slot++;
22  }

```

第三段代码来自 record/RecordPage.c 的 RecordPageInsertAfter。它复用 RecordPageSearchAfter 查找空 slot，找到后只需要把 flag 改成已使用。

代码 6-5 RecordPageInsertAfter 查找并占用空 slot

```

1  int RecordPageInsertAfter(RecordPage *recordPage, int slot){
2      int newSlot = RecordPageSearchAfter(recordPage, slot,
3          RECORD_PAGE_EMPTY);
4      if(newSlot >= 0){
5          RecordPageSetFlag(recordPage, newSlot, RECORD_PAGE_USED);
6      }
7      return newSlot;
8  }

```

第四段代码来自 record/TableScan.c 的 TableScanInit。它把表名转换为表文件名，并根据文件是否为空决定追加新块还是打开已有块。

代码 6-6 TableScanInit 建立表文件扫描

```

1  CString *filename = CStringCreateFromCString(TBName);
2  CStringAppendCStr(filename, ".tbl");
3  tableScan->fileName = filename;
4
5  tableScan->recordPage = NULL;
6  if(TransactionSize(transaction, filename) == 0){
7      TableScanMoveToNewBlock(tableScan);
8  }else{
9      TableScanMoveToBlock(tableScan, 0);
10 }

```

6.11 运行与观察

在 `D:\textbackslash code\textbackslash DBMS\textbackslash NewDBMS\textbackslash DBMS_C` 下使用固定命令构建并运行测试：

代码 6-7 构建并运行 Record 测试

```
1 cmake -S . -B build
2 mingw32-make -C build -j8 SHELL=cmd.exe
3 ctest --test-dir build --output-on-failure -R Record
```

观察测试输出时，可以重点看五个点：

1. Schema 添加字段后，字段类型和长度是否与调用参数一致。
2. Layout 是否把第一个字段放在 slot 标志位之后。
3. RID 是否只承担“块号 + 槽号”的轻量位置标识职责。
4. RecordPage 写入字段后，是否能从同一个 slot 读回相同值。
5. TableScan 是否能在表文件中插入一条记录，并通过顺序扫描读回。

6.12 思考题

1. 为什么记录页中的每个 slot 需要一个状态标志位？
2. Layout 为什么要把字段名映射到页内 offset，而不是每次读写时重新计算？
3. RID 为什么只保存 block number 和 slot，而不直接保存字段值或表结构？
4. TableScan 为什么需要建立在 RecordPage 之上，而不是直接操作 Page？

第 7 章 元数据管理实验

7.1 实验目的

本章用于理解 DBMS_C 如何保存和读取“关于表的数据”。记录管理层已经能把字段和记录放进表文件，但数据库系统还需要知道有哪些表、每个表有哪些字段、视图定义放在哪里、索引登记在哪张目录表中。元数据管理层的任务就是维护这些系统目录，使上层模块可以通过表名、字段名、视图名和索引名重新找到结构信息。

本章仍然遵循“先补真实 CTest，再写章节”的方式。本轮新增的 MetadataBasicTest 只验证元数据登记和读回能力：创建表后读回 Layout，创建视图后读回 view definition，创建索引后从 idxcat 读回 IndexInfo。不测试 SQL Parser，不测试查询计划选择索引，也不测试优化器代价模型。

7.2 模块作用

metadata/ 位于记录管理和 SQL 执行层之间。它并不直接执行查询，而是提供系统目录访问能力。上层在执行 CREATE TABLE、CREATE VIEW、CREATE INDEX 时，需要把定义写入 catalog；后续查询、更新和计划生成阶段，再通过 MetadataManager 查回表布局、视图定义、索引信息和统计信息。

当前实现中，MetadataMgr 是一个外观对象，内部持有 TableManager、ViewManager、StatManager 和 IndexManager。调用者通常不需要分别创建这些子管理器，而是通过 MetadataManager 的接口完成元数据创建与查询。

7.3 相关源码文件

表 7-1 列出本章直接涉及的真实源码文件。建议先看 MetadataManager.h，了解对外接口；再看 TableManager.c，因为系统目录的核心表 tblcat 和 fldcat 都由它创建和读取；最后再看 View、Index 和 Stat 相关文件。

7.4 MetadataManager 与子管理器

图 7-1 展示了 MetadataMgr 和各子管理器的关系。MetadataManager 本身不保存所有 catalog 记录，而是把不同类型的元数据操作分派给对应子管理器。

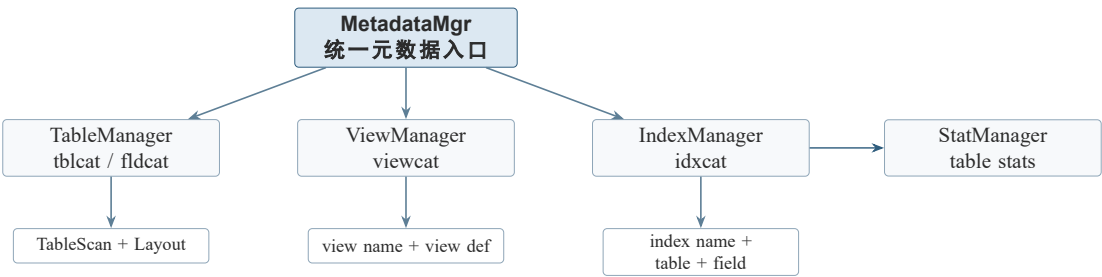


图 7-1 MetadataMgr 与 Table/View/Index/Stat 子管理器的关系

表 7-1 Metadata 模块相关源码文件

文件	本章关注点
metadata/MetadataManager.h, metadata/MetadataManager.c	封装表、视图、索引、统计信息管理器，对外提供统一元数据接口。
metadata/TableManager.h, metadata/TableManager.c	创建和读取 tblcat、fldcat，并根据 catalog 恢复表 Layout。
metadata/ViewManager.h, metadata/ViewManager.c	创建 viewcat，登记并读取视图定义字符串。
metadata/IndexManager.h, metadata/IndexManager.c	创建 idxcat，登记索引名、表名和字段名，并返回 IndexInfo map。
metadata/IndexInfo.h, metadata/IndexInfo.c	保存单个索引的字段、布局 and 统计信息，并提供估算接口。
metadata/StatManager.h, metadata/StatManager.c	基于表扫描计算表的块数和记录数等简单统计信息。
metadata/StatInfo.h, metadata/StatInfo.c	保存统计信息值，并提供访问函数。
record/Schema.h, record/Layout.h	元数据读回后重新构造表字段集合和字段偏移。
tx/Transaction.h	catalog 表通过事务和 TableScan 写入底层表文件。

初始化 MetadataManager 时，代码会按顺序创建 TableManager、ViewManager、StatManager 和 IndexManager。若当前数据库是新建的，TableManager 会先建立 tblcat 与 fldcat，ViewManager 会建立 viewcat，IndexManager 会建立 idxcat。这些 catalog 本质上仍然是普通表文件，只是由系统内部使用。

7.5 系统目录表

当前实现中主要有四类系统目录表，如图 7-2 所示。tblcat 记录表名和 slot 大小，fldcat 记录每个字段的类型、长度和 offset；这两张表合在一起可以恢复一张用户表的 Layout。viewcat 保存视图名和视图定义字符串，idxcat 保存索引名、表名和字段名。

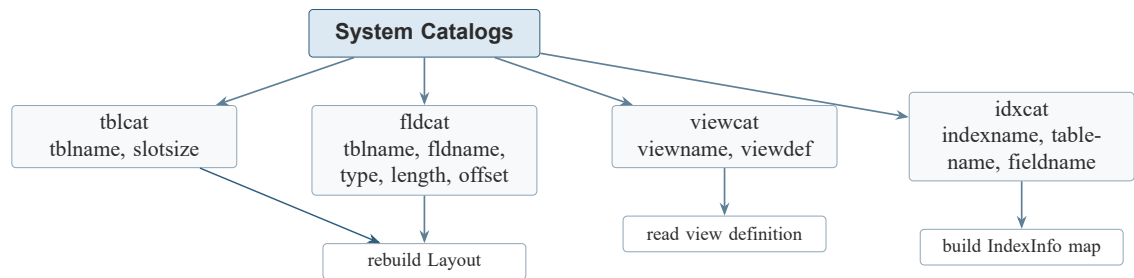


图 7-2 tblcat、fldcat、viewcat、idxcat 的系统目录关系

这些系统目录的一个重要特点是“用表管理表”。目录项不是写在特殊配置文件中，而是通过已有的 Schema、Layout、TableScan、Transaction 链路写入表文件。这种设计让元数据层可以复用记录管理和事务层能力。

7.6 CREATE TABLE 元数据登记过程

创建表时，MetadataManager 会把请求转发到 TableManager。TableManager 首先根据用户传入的 Schema 生成 Layout，然后向 tblcat 插入一条记录，保存表名和 slot size；接着遍历用户 Schema 中的每个字段，向 fldcat 插入字段记录，保存字段名、类型、长度和 offset。图 7-3 展示了这条调用链。

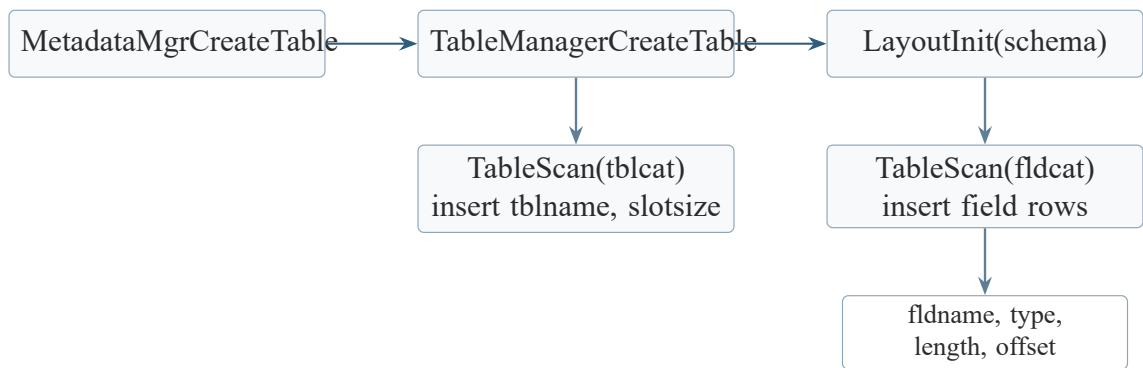


图 7-3 CREATE TABLE 元数据登记调用链

读取表布局时，TableManagerGetLayout 会反向扫描 catalog：先在 tblcat 中找到目标表的 slot size，再在 fldcat 中收集该表所有字段，重建 Schema 和 offset map，最后构造新的 Layout 返回给调用者。

7.7 配套测试

本章新增并注册了配套测试文件：

- test/metadata/MetadataBasicTest.c

该测试使用独立数据库目录，目录名包含进程号和用例计数；日志文件为 metadata_basic.log。测试结束时会尽量清理 tblcat.tbl、fldcat.tbl、viewcat.tbl、idxcat.tbl 以及本章创建的测试表文件。

表 7-2 MetadataBasicTest 覆盖内容

测试函数	验证目标
test_metadata_manager_create_table_and_read_layout	初始化 MetadataMgr，创建 student_meta 表，再通过 MetadataMgrGetLayout 读回 Schema、字段类型、长度和 offset。
test_metadata_manager_create_view_and_read_definition	创建 student_view，并通过 MetadataMgrGetViewDef 读回相同 view definition。
test_metadata_manager_create_index_and_read_index_info	创建 student_idx_meta 表和 idx_student_id 索引，再通过 MetadataManagerGetIndexInfo 验证 idxcat 登记结果。

单独运行本章测试可以使用：

代码 7-1 运行 Metadata 相关测试

```
1 ctest --test-dir build --output-on-failure -R Metadata
```

只运行本章新增测试可以使用：

代码 7-2 只运行 MetadataBasicTest

```
1 ctest --test-dir build --output-on-failure -R MetadataBasicTest
```

7.8 关键代码讲解

第一段代码来自 metadata/MetadataManager.c 的 MetadataMgrInit。它展示了 MetadataManager 如何组合四个子管理器。

代码 7-3 MetadataMgrInit 组合子管理器

```

1 MetadataMgr* MetadataMgrInit(bool isNew, Transaction *tx) {
2     MetadataMgr *mdMgr = (MetadataMgr*)malloc(sizeof(MetadataMgr));
3     mdMgr->tblMgr = TableManagerInit(isNew, tx);
4     mdMgr->viewMgr = ViewManagerInit(isNew, mdMgr->tblMgr, tx);
5     mdMgr->statMgr = StatManagerInit(mdMgr->tblMgr, tx);
6     mdMgr->idxMgr = IndexMgrInit(isNew, mdMgr->tblMgr, mdMgr->statMgr, tx);
7     return mdMgr;
8 }

```

第二段代码来自 metadata/TableManager.c 的 TableManagerCreateTable。它先把表级信息写入 tblcat，再把字段级信息逐行写入 fldcat。

代码 7-4 TableManagerCreateTable 写入 tblcat 与 fldcat

```

1 TableScan *table = TableScanInit(transaction, tblcatParam,
2                                     tableManager->tableCatalogLayout);
3 Scan *tableCatalog = ScanInit(table, SCAN_TABLE_CODE);
4 TableScanInsert(tableCatalog);
5 TableScanSetString(tableCatalog, tblnameParam, tblname);
6 TableScanSetInt(tableCatalog, slotsizeParam, layout->SlotSize);
7 TableScanClose(tableCatalog);
8
9 TableScan *field = TableScanInit(transaction, fldcatParam,
10                                    tableManager->fieldCatalogLayout);
11 Scan *fieldCatalog = ScanInit(field, SCAN_TABLE_CODE);
12 FieldNode *fieldNode = sch->fields;
13 while(fieldNode){
14     CString *fldname = fieldNode->fileName;
15     TableScanInsert(fieldCatalog);
16     TableScanSetString(fieldCatalog, tblnameParam, tblname);
17     TableScanSetString(fieldCatalog, fldnameParam, fldname);
18     TableScanSetInt(fieldCatalog, typeParam, SchemaType(sch, fldname));
19     TableScanSetInt(fieldCatalog, lengthParam, SchemaLength(sch, fldname));
20     TableScanSetInt(fieldCatalog, offsetParam, LayoutOffset(layout, fldname));
21     fieldNode = fieldNode->next;
22 }

```

第三段代码来自 metadata/ViewManager.c。视图定义被作为普通字符串写入 viewcat，读取时按 view name 查找并复制定义字符串返回。

代码 7-5 ViewManagerCreateView 登记视图定义

```

1 Layout *layout = TableManagerGetLayout(viewManager->tableManager,
2                                     viewcatTbl, transaction);
3 TableScan *scan = TableScanInit(transaction, viewcatTbl, layout);
4 Scan *tableScan = ScanInit(scan, SCAN_TABLE_CODE);
5 TableScanInsert(tableScan);
6 TableScanSetString(tableScan, viewnameParam, vname);
7 TableScanSetString(tableScan, viewdefParam, vdef);
8 TableScanClose(tableScan);

```

第四段代码来自 `metadata/IndexManager.c`。索引登记只保存索引名、表名和字段名；本章不测试 `HashIndex` 的数据结构，也不测试 `Plan` 是否选择索引。

代码 7-6 IndexMgrCreateIndex 写入 idxcat

```

1 TableScan *ts = TableScanInit(tx, idxcatTbl, indexMgr->layout);
2 Scan *scan = ScanInit(ts, SCAN_TABLE_CODE);
3 TableScanInsert(scan);
4 TableScanSetString(scan, indexnameParam, idxname);
5 TableScanSetString(scan, tablenameParam, tblname);
6 TableScanSetString(scan, fieldnameParam, fldname);
7 TableScanClose(scan);

```

7.9 运行与观察

在 `D:\textbackslash code\textbackslash DBMS\textbackslash NewDBMS\textbackslash DBMS_C` 下使用固定命令构建并运行测试：

代码 7-7 构建并运行 Metadata 测试

```

1 cmake -S . -B build
2 mingw32-make -C build -j8 SHELL=cmd.exe
3 ctest --test-dir build --output-on-failure -R Metadata

```

观察测试输出时，可以重点看五个点：

1. `MetadataMgrInit(true, tx)` 是否创建四个子管理器。
2. 创建表后，`MetadataMgrGetLayout` 是否能读回字段类型、长度和 `offset`。
3. 视图定义是否能够通过 `viewcat` 原样读回。
4. 索引登记是否能够通过 `idxcat` 转换为 `IndexInfo`。
5. 本章测试是否只验证 `catalog` 行为，而没有提前进入 `Parser`、`Plan` 或查询优化。

7.10 思考题

1. 为什么数据库需要系统目录表，而不是只把 Schema 保存在内存结构中？
2. `tblcat` 和 `fldcat` 为什么要拆成两张目录表？
3. 当前 `viewcat` 只保存视图定义字符串，这种设计的优点和局限是什么？
4. `idxcat` 登记索引后，为什么还需要后续 Plan 章节来说明索引如何被使用？

第 8 章 SQL 解析实验

8.1 实验目的

本章用于理解 DBMS_C 如何把一条 SQL 字符串转换为后续模块可以使用的结构化对象。对于数据库系统来说，SQL 输入只是普通文本；在进入查询计划、扫描执行或更新执行之前，系统必须先完成词法切分、语法识别和 Data 对象构造。本章只讨论解析阶段，不执行 SQL，不生成 Plan，也不访问表数据。

本章继续采用“先补真实 CTest，再写章节”的实验组织方式。本轮新增并验证通过的测试是 test/parse/ParseBasicTest.c，覆盖 SELECT、INSERT、UPDATE、DELETE、CREATE TABLE、CREATE VIEW 和 CREATE INDEX 的基础解析行为。测试只断言 Parser 产出的 Data 对象是否符合输入 SQL，不进入第 9 章的 Plan/Scan，也不进入第 10 章的更新执行。

8.2 SQL 解析在 DBMS_C 中的位置

SQL 解析位于命令行输入和执行模块之间。用户在 CLI 中输入 SQL 后，系统先通过 Lexer 将字符流切分为关键字、标识符、常量和分隔符，再由 Parser 按 SQL 类型构造对应的数据对象。例如 SELECT 会生成 QueryData，INSERT 会生成 InsertData，CREATE TABLE 会生成 CreateTableData。这些 Data 对象本身不负责执行，只负责保存“解析结果”。

图 8-1 展示了从 SQL 字符串到 Data 对象的转换关系。图中右侧的 Plan、UpdatePlanner 和 Transaction 只是后续消费者，本章不会展开它们的执行逻辑。

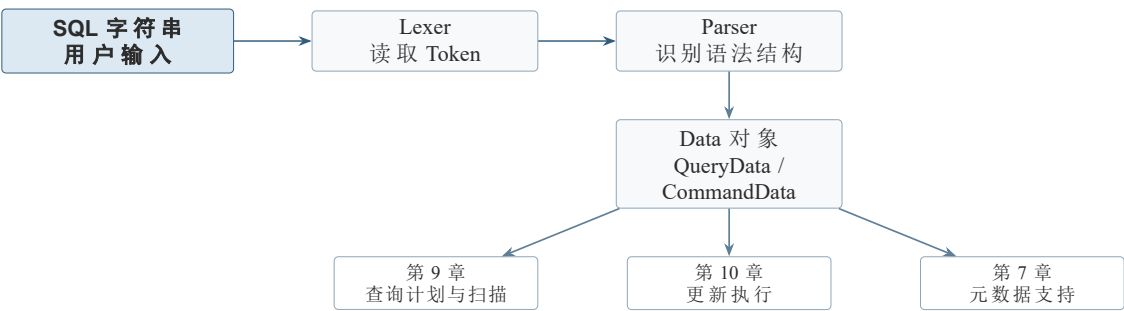


图 8-1 SQL 字符串到 Data 对象的转换关系

8.3 相关源码文件

表 8-1 列出本章直接涉及的真实源码文件。阅读时建议先看 Lexer.h 和 Parser.h，确认对外 API；再看 Parser.c，理解不同 SQL 命令如何被分发；最后结合各 Data 对象头文件观察解析结果保存在哪里。

表 8-1 Parse 模块相关源码文件

文件	本章关注点
parse/Lexer.h, parse/Lexer.c	基于 StreamTokenizer 读取关键字、标识符、整数常量、字符串常量和分隔符。
parse/Parser.h, parse/Parser.c	SQL 解析主入口，构造 QueryData 或 CommandData。
parse/PredParser.h, parse/PredParser.c	谓词解析相关结构；当前主要逻辑在 Parser 的 ParserPredicate、ParserTerm 中体现。
parse/QueryData.h, parse/QueryData.c	保存 SELECT 字段列表、表名列表和谓词。
parse/InsertData.h, parse/InsertData.c	保存 INSERT 的表名、字段列表和值列表。
parse/ModifyData.h, parse/ModifyData.c	保存 UPDATE 的表名、目标字段、新值表达式和谓词。
parse/DeleteData.h, parse/DeleteData.c	保存 DELETE 的表名和谓词。
parse/CreateTableData.h, parse/CreateTableData.c	保存 CREATE TABLE 的表名和 Schema。
parse/CreateViewData.h, parse/CreateViewData.c	保存 CREATE VIEW 的视图名和内部 SELECT 查询定义。
parse/CreateIndexData.h, parse/CreateIndexData.c	保存 CREATE INDEX 的索引名、表名和字段名。
query/Predicate.h, query/Term.h, query/Expression.h	表示 WHERE 条件中的谓词、比较项和表达式。

8.4 Lexer、Parser 与 PredParser 的职责

Lexer 的职责是面向 Token：它不理解整条 SQL 的业务含义，只负责判断当前 Token 是否匹配某类符号，并在需要时消费它。Parser 的职责是面向语法结构：它按照 SQL 命令格式调用 Lexer，逐步构造字段列表、表名列、常量列表、表达式、谓词和命令 Data 对象。PredParser 当前只保留了谓词解析结构，实际谓词构造主要通过 Parser 中的 ParserPredicate 和 ParserTerm 完成。

图 8-2 展示三者的关系。注意，本章只观察 Parser 产物，不尝试让 Parser 执行 SQL。

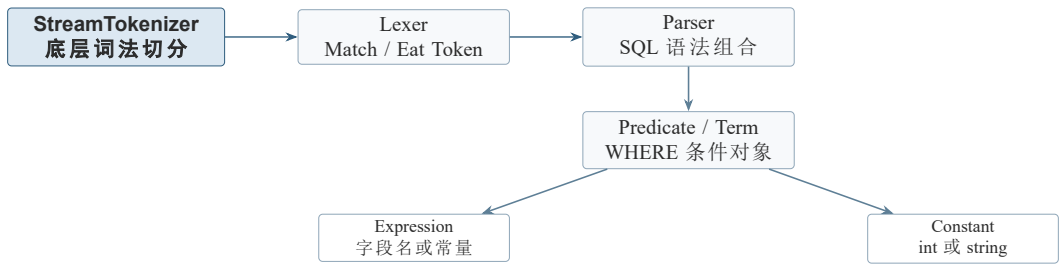


图 8-2 Lexer、Parser 与谓词对象的关系

8.5 不同 SQL 命令的解析结果

Parser 对 SELECT 和更新类命令使用不同入口。SELECT 通过 ParserQuery 返回 QueryData；INSERT、DELETE、UPDATE 和 CREATE 族命令通过 ParserUpdateCmd 返回 CommandData。表 8-2 汇总了本章测试覆盖的 SQL 与 Data 对象映射。

表 8-2 SQL 命令到 Data 对象的映射

SQL 类型	解析结果	保存的核心信息
SELECT	QueryData	fields、tables、predicate
INSERT	CommandData + InsertData	表名、字段列表、常量值列表
UPDATE	CommandData + ModifyData	表名、目标字段、新值表达式、谓词
DELETE	CommandData + DeleteData	表名、谓词
CREATE TABLE	CommandData + CreateTableData	表名、Schema 字段类型和长度
CREATE VIEW	CommandData + CreateViewData	视图名、内部 SELECT 的 QueryData
CREATE INDEX	CommandData + CreateIndexData	索引名、表名、字段名

8.6 配套测试

本章新增并注册了配套测试文件：

- test/parse/ParseBasicTest.c

该测试不创建数据库目录，也不访问文件、记录、元数据或执行器。它只构造 Parser，输入短 SQL 字符串，然后断言返回的结构体字段符合预期。表 8-3 列出每个测试函数

的覆盖内容。

表 8-3 ParseBasicTest 覆盖内容

测试函数	主要断言
test_parse_select_fields_tables_and_predicate	QueryData 非空；字段列表为 id,name；表名为 student；谓词为 id = 1。
test_parse_insert_table_fields_and_values	CommandData.code 为 CMD_INSERT_DATA；表名、字段数量、值数量、整数字面量和字符串字面量正确。
test_parse_update_target_value_and_predicate	CMD_MODIFY_DATA；表名为 student；目标字段为 name；新值为字符串常量；谓词为 id = 1。
test_parse_delete_table_and_predicate	CMD_DELETE_DATA；表名为 student；谓词对象非空且表示 id = 1。
test_parse_create_table_schema	CMD_CREATE_TABLE；表名为 student；Schema 含 id int 和 name varchar(20)。
test_parse_create_view_query_data	CMD_CREATE_VIEW；视图名正确；内部 QueryData 的字段、表名和谓词正确。
test_parse_create_index_identifiers	CMD_CREATE_INDEX；索引名、表名和字段名正确。

本章测试已经在临时验证目录 build-check 中通过。单独运行新增测试的命令如下：

代码 8-1 运行 ParseBasicTest

```
1 ctest --test-dir build-check --output-on-failure -R ParseBasicTest
```

运行 Parse 相关测试的命令如下：

代码 8-2 运行 Parse 相关测试

```
1 ctest --test-dir build-check --output-on-failure -R Parse
```

8.7 关键代码讲解

第一段代码来自 `parse/Parser.c` 的 `ParserQuery`。它说明 `SELECT` 语句如何被拆成字段列表、表名列表和可选 `WHERE` 谓词。

代码 8-3 ParserQuery 构造 QueryData

```

1 QueryData *ParserQuery(Parser*parser){
2     LexerEatKeyWord(parser->lexer,"select");
3     CList *fields = ParserSelectList(parser);
4     LexerEatKeyWord(parser->lexer,"from");
5     CList*tables = ParserTabletList(parser);
6     Predicate *predicate = PredicateInit(NULL);
7     if(LexerMatchKeyWord(parser->lexer,"where")){
8         LexerEatKeyWord(parser->lexer,"where");
9         predicate = ParserPredicate(parser);
10    }
11    return QueryDataInit(fields,tables,predicate);
12 }

```

第二段代码来自 `parse/Parser.c` 的 `ParserUpdateCmd`。它是更新类和 `CREATE` 类命令的统分发入口，根据当前关键字选择具体解析函数。

代码 8-4 ParserUpdateCmd 分发更新类命令

```

1 CommandData *ParserUpdateCmd(Parser*parser){
2     if(LexerMatchKeyWord(parser->lexer,"insert")){
3         return ParserInsert(parser);
4     }else if(LexerMatchKeyWord(parser->lexer,"delete")){
5         return ParserDelete(parser);
6     }else if(LexerMatchKeyWord(parser->lexer,"update")){
7         return ParserModify(parser);
8     }else{
9         return ParserCreate(parser);
10    }
11 }

```

第三段代码来自 `parse/Parser.c` 的 `ParserTerm` 和 `ParserPredicate`。当前实现把一个比较表达式封装为 `Term`；如果遇到 `and`，再把后续 `Predicate` 合并进来。

代码 8-5 ParserTerm 与 ParserPredicate 构造 WHERE 条件

```

1 Term *ParserTerm(Parser*parser){
2     Expression *lhs = ParserExpression(parser);
3     const CompareOp op = LexerGetCurrentOp(parser->lexer);

```

```

4     LexerNextToken(parser->lexer);
5     Expression *rhs = ParserExpression(parser);
6     return TermInit(lhs,rhs,op);
7 }
8
9 Predicate *ParserPredicate(Parser*parser){
10    Predicate * predicate = PredicateInit(ParserTerm(parser));
11    if(LexerMatchKeyWord(parser->lexer,"and")){
12        LexerEatKeyWord(parser->lexer,"and");
13        PredicateConjoinWith(predicate, ParserPredicate(parser));
14    }
15    return predicate;
16 }

```

第四段代码来自 `parse/Parser.c` 的 `ParserCreateTable`。它把字段定义列表解析为 `Schema`，再包装进 `CreateTableData`。

代码 8-6 ParserCreateTable 构造 CreateTableData

```

1 CommandData* ParserCreateTable(Parser*parser){
2     LexerEatKeyWord(parser->lexer,"table");
3     CString *tblname = CStringCreateFromCStr(LexerEatId(parser->lexer));
4     LexerEatDelim(parser->lexer,'(');
5     Schema *schema = ParserFieldDefs(parser);
6     LexerEatDelim(parser->lexer,')');
7
8     CommandData * commandData = malloc(sizeof(CommandData));
9     commandData->code = CMD_CREATE_TABLE;
10    commandData->data.createTableData = CreateTableDataInit(tblname,schema);
11    return commandData;
12 }

```

8.8 运行与观察

本章观察重点不是 SQL 是否能产生结果，而是解析出的结构体是否正确。运行 `ParseBasicTest` 时，可以重点观察以下几个方面：

1. SELECT 是否被解析为 `QueryData`，并保存字段列表、表名列表和谓词对象。
2. INSERT、UPDATE、DELETE、CREATE 族命令是否被统一包装为 `CommandData`，并设置正确的 `CommandCode`。
3. 字符串常量和整数常量是否被区分为不同的 `Constant`。
4. `id = 1` 这样的 WHERE 条件是否被表示为 `Predicate -> Term -> Expression/Constant`。
5. CREATE TABLE 是否生成真实 `Schema`，并能读出字段类型和长度。

图 8-3 用 INSERT 和 CREATE TABLE 为例说明解析阶段的调用链。它只展示 Data 对象生成，不展示后续执行。

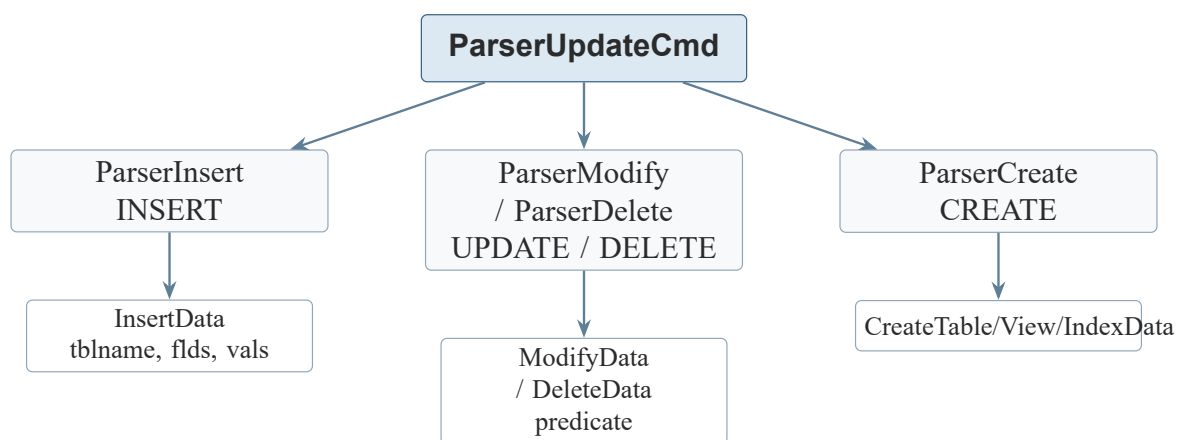


图 8-3 更新类 SQL 命令到 Data 对象的解析调用链

8.9 思考题

1. 为什么 Parser 不直接执行 SQL，而是先生成 QueryData 或 CommandData？
2. LexerMatchKeyword 和 LexerEatKeyword 的区别是什么？如果把二者混用，会产生什么问题？
3. WHERE 条件为什么要拆成 Predicate、Term 和 Expression 三层对象？
4. CREATE TABLE 解析阶段为什么只生成 Schema，而不直接创建表文件？

第 9 章 查询计划与扫描执行实验

9.1 实验目的

本章用于理解 SQL 解析之后，DBMS_C 如何把 QueryData 转换为查询计划，再由查询计划打开为运行期扫描器，最终逐条读取满足条件的记录。第 8 章已经说明 Parser 只负责把 SQL 文本转换为结构化 Data 对象；本章继续向后走一步，观察 SELECT 查询如何经过 BasicQueryPlanner 生成 Plan，再由 Plan.open 生成 Scan。

本章仍然只讨论 SELECT 查询主链路，不展开 INSERT、UPDATE、DELETE、CREATE TABLE、CREATE VIEW、CREATE INDEX 等更新执行路径。那些命令会在第 10 章“更新执行实验”中单独讨论。

9.2 Plan/Scan 在 SQL 执行链路中的位置

Plan/Scan 位于 SQL 解析和实际记录访问之间。Parser 产出的 QueryData 只说明“查哪些字段、查哪些表、有什么条件”；BasicQueryPlanner 根据这些信息构造 Plan 树；当执行阶段真正需要读取数据时，再调用 Plan 的 open 函数生成 Scan 链。Scan 链负责逐条推进记录，并通过 next/getInt/getString/getVal 等接口取出当前记录字段。

图 9-1 展示本章在主链路中的位置。注意，本章的测试会直接准备表和记录，然后通过 Planner 执行 SELECT 查询验证结果。

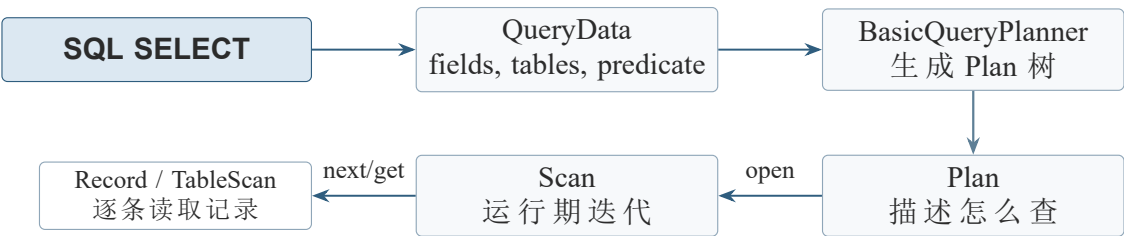


图 9-1 Plan/Scan 在 SELECT 执行链路中的位置

9.3 模块作用

BasicQueryPlanner 是当前基础查询计划器。它接收 QueryData，为每个表创建 TablePlan；如果存在多张表，则用 ProductPlan 组合；随后用 SelectPlan 包装谓词筛选；最后用 ProjectPlan 包装字段投影。

Plan 表示尚未执行的查询计划，核心是函数指针集合和类型码。不同 Plan 通过统一接口提供 open、blocksAccessed、recordsOutput、distinctValues 和 schema。其中 open 是从计划进入运行期扫描的关键入口。

Scan 表示运行期记录迭代器。它同样通过统一函数指针隐藏不同扫描器的差异，例如 TableScan 访问真实表记录，SelectScan 过滤记录，ProjectScan 限制可见字段，ProductScan 组合两路扫描结果。

9.4 相关源码文件

表 9-1 列出本章直接涉及的真实源码文件。建议先阅读 `Planner.c` 和 `BasicQueryPlanner.c`，再阅读各类 `Plan` 的 `Open` 函数，最后观察 `Scan` 层如何通过统一接口向下转发。

表 9-1 Plan/Scan 相关源码文件

文件	本章关注点
<code>plan/Planner.h</code> , <code>plan/Planner.c</code>	查询计划入口 <code>PlannerCreateQueryPlan</code> ，把 SQL 文本解析为 <code>QueryData</code> 并转交查询计划器。
<code>plan/BasicQueryPlanner.h</code> , <code>plan/BasicQueryPlanner.c</code>	根据 <code>QueryData</code> 构造 <code>TablePlan</code> 、 <code>ProductPlan</code> 、 <code>SelectPlan</code> 、 <code>ProjectPlan</code> 。
<code>plan/Plan.h</code> , <code>plan/Plan.c</code>	<code>Plan</code> 类型码、 <code>union</code> 和 <code>open/schema</code> 等统一函数指针。
<code>plan/TablePlan.h</code> , <code>plan/TablePlan.c</code>	单表访问计划，打开后生成 <code>TableScan</code> 包装。
<code>plan/SelectPlan.h</code> , <code>plan/SelectPlan.c</code>	条件选择计划，打开后生成 <code>SelectScan</code> 。
<code>plan/ProjectPlan.h</code> , <code>plan/ProjectPlan.c</code>	字段投影计划，打开后生成 <code>ProjectScan</code> 。
<code>plan/ProductPlan.h</code> , <code>plan/ProductPlan.c</code>	多表乘积计划，打开后生成 <code>ProductScan</code> 。
<code>query/Scan.h</code> , <code>query/Scan.c</code>	<code>Scan</code> 统一接口和不同扫描器的函数指针绑定。
<code>query/SelectScan.c</code> , <code>query/ProjectScan.c</code> , <code>query/ProductScan.c</code>	运行期筛选、投影、多表组合扫描逻辑。
<code>query/Predicate.c</code> , <code>query/Term.c</code> , <code>query/Expression.c</code> , <code>query/Constant.c</code>	<code>WHERE</code> 条件判断所需的谓词、表达式和常量逻辑。
<code>record/TableScan.h</code> , <code>record/TableScan.c</code>	表记录扫描，提供真实记录的插入、 <code>next</code> 、 <code>getInt</code> 、 <code>getString</code> 等接口。

9.5 Plan 与 Scan 的区别

`Plan` 和 `Scan` 是本章最重要的一组概念。`Plan` 更像“查询方案”，描述要访问哪些表、如何筛选、如何投影，以及估算信息；`Scan` 更像“执行游标”，已经绑定到底层记录，能够逐条向前推进。表 9-2 对比二者职责。

9.6 SELECT 查询计划生成流程

以本章测试中的 SQL 为例：

表 9-2 Plan 与 Scan 职责对照

对象	职责	典型接口
Plan	描述“怎么查”，尚未逐条读取记录；可以暴露 schema 和估算信息。	open、schema、recordsOutput、blocksAccessed
Scan	执行“正在查”，通过游标式接口逐条推进并读取字段值。	next、getInt、getString、getVal、hasField

代码 9-1 本章 SELECT 示例

```
1 select id, name from student_plan where id = 1
```

该语句经过 Parser 后得到 QueryData，其中字段列表为 id,name，表名列表为 student_plan，谓词为 id = 1。随后 BasicQueryPlannerCreatPlan 按照图 9-2 的顺序构造 Plan 树。

打开 Plan 后，会形成对应的 Scan 链。图 9-3 展示了当前测试中最外层 ProjectPlan 打开后如何逐层调用内部 Plan 的 open 函数。测试不直接访问 ProjectScan 或 SelectScan 的内部字段，而是通过公开的 Scan 接口验证结果。

图 9-4 进一步把测试准备、计划生成、扫描执行串起来。测试先创建 student_plan(id int, name varchar(20))，直接通过 TableScan 插入两条记录，然后用 Planner 查询 id = 1 的记录。这里直接使用 TableScan 准备数据，是为了避免提前使用第 10 章才讨论的 INSERT 执行路径。

9.7 配套测试

本章新增并注册了配套测试文件：

- test/plan/PlanScanBasicTest.c

测试使用独立数据库目录，目录名以 plan_scan_basic_test_db 开头，并包含进程号和用例计数。每个用例创建 FileManager、LogManager、BufferManager、Transaction 和 MetadataManager；随后创建表 student_plan(id int, name varchar(20))，通过 TableScan 插入 (1, "wang") 和 (2, "li")。查询语句只选择 id = 1，因此期望只读回第一条记录。

本章测试已在 build-plancheck 中验证通过。单独运行本章测试：

代码 9-2 运行 PlanScanBasicTest

```
1 ctest --test-dir build-plancheck --output-on-failure -R PlanScanBasicTest
```

运行 Plan 相关测试：

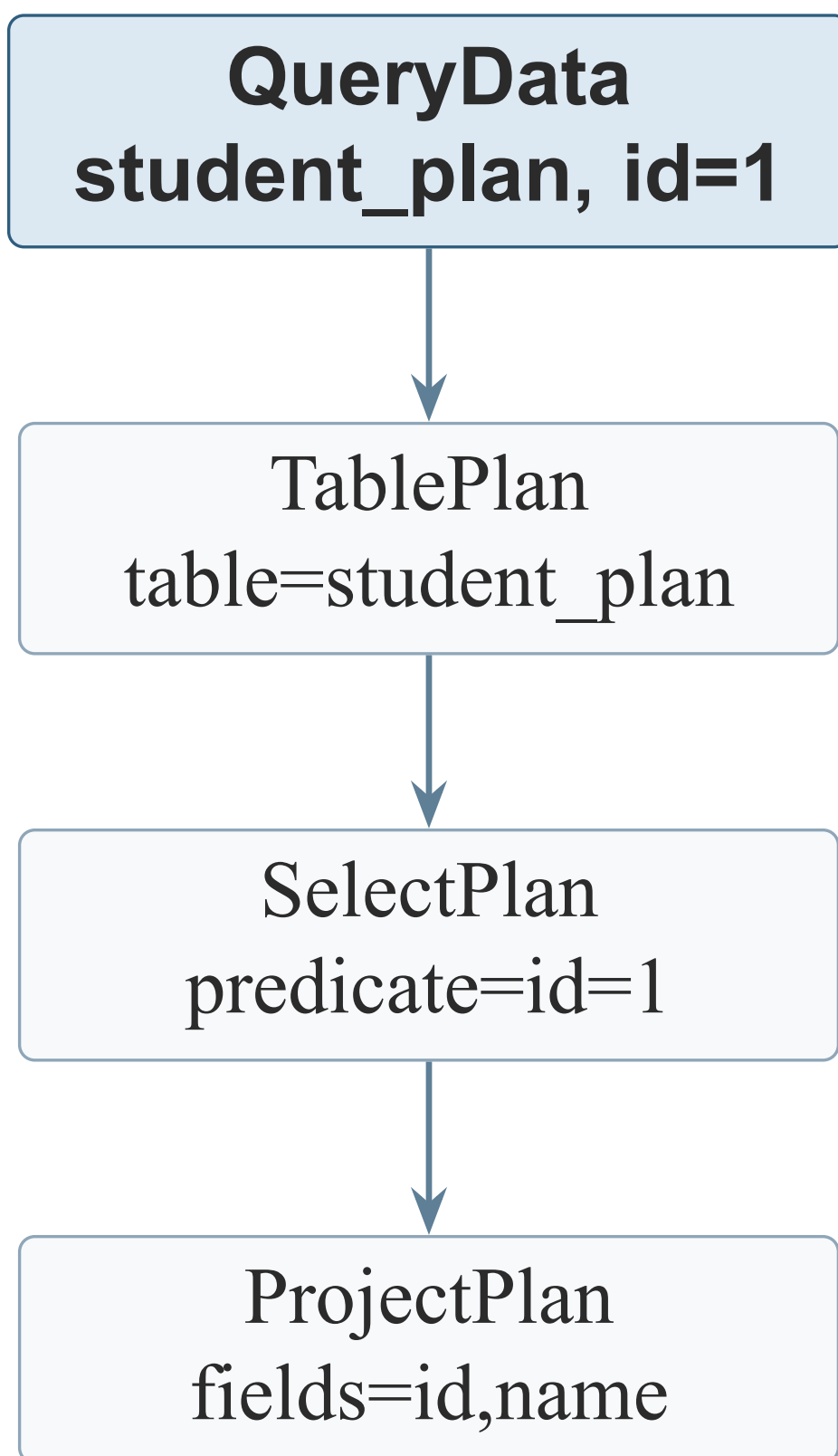


图 9-2 QueryData 到 Plan 树的结构

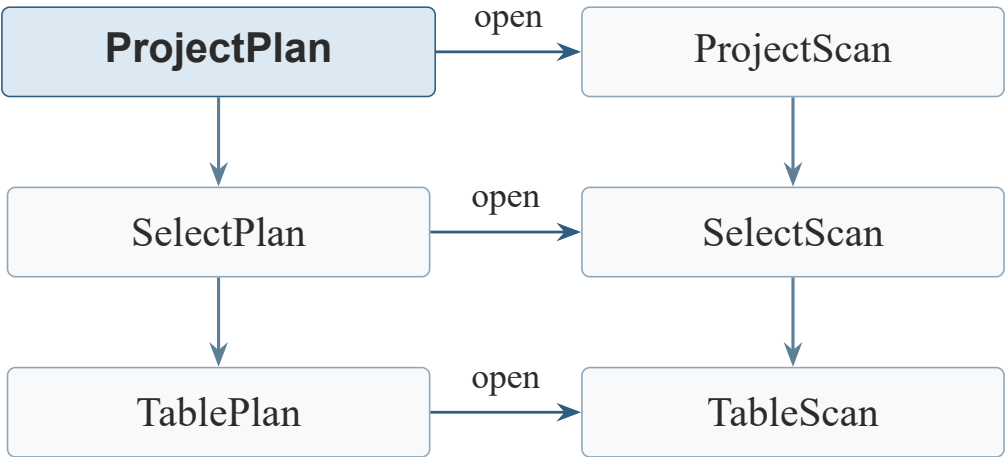


图 9-3 Plan 树到 Scan 链的映射

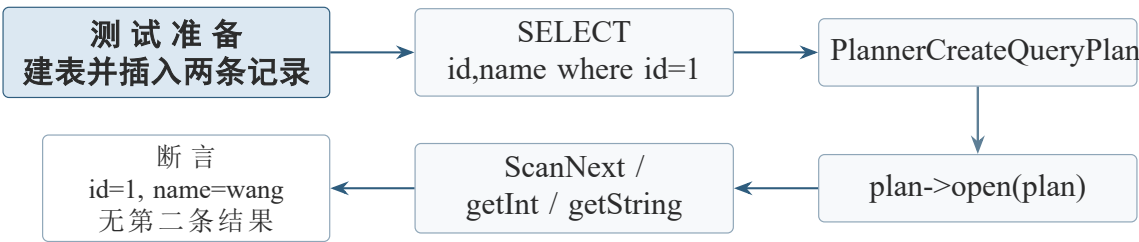


图 9-4 SELECT 示例执行链路

表 9-3 PlanScanBasicTest 覆盖内容

测试函数	主要断言
test_basic_query_planner_select_project_scan	断言 Plan 非空；最外层 Plan 为 PLAN_PROJECT_CODE；内部依次为 PLAN_SELECT_CODE 和 PLAN_TABLE_CODE；open 后 Scan 非空且最外层为 SCAN_PROJECT_CODE；通过公开 Scan 接口读回 id=1,name=wang，并断言没有第二条匹配结果。
test_plan_open_returns_scan_for_query_result	断言 Plan 非空；open 返回 Scan 非空；Scan 最外层为 SCAN_PROJECT_CODE；通过 hasField/next/getInt/getString 验证查询结果，不访问 ProjectScan 或 SelectScan 内部字段。

代码 9-3 运行 Plan 相关测试

```
1 ctest --test-dir build-plancheck --output-on-failure -R Plan
```

9.8 关键代码讲解

第一段代码来自 plan/Planner.c 的 PlannerCreateQueryPlan。它先调用 Parser 得到 QueryData，再把 QueryData 交给基础查询计划器。

代码 9-4 PlannerCreateQueryPlan 查询计划入口

```
1 Plan* PlannerCreateQueryPlan(Planner*planner,CString*qry,Transaction*transaction){
2     Parser *parser = ParserInit(CStringGetPtr(qry));
3     QueryData*data = ParserQuery(parser);
4     QueryDataTrace(data);
5     return BasicQueryPlannerCreatPlan(planner->queryPlanner,data,transaction);
6 }
```

第二段代码来自 plan/BasicQueryPlanner.c。当前基础计划器为每张表创建 TablePlan，多表时用 ProductPlan 合并，然后统一包上 SelectPlan 和 ProjectPlan。

代码 9-5 BasicQueryPlannerCreatPlan 的计划包装顺序

```
1 Plan *p = CListRemoveByIndex(plans,0);
2 CListNode *head = plans->head;
3 while(head){
4     Plan*nextPlan = ((Plan *)head->data);
5     ProductPlan *productPlan = ProductPlanInit(p,nextPlan);
6     p = PlanInit(productPlan,PLAN_PRODUCT_CODE);
7     head=head->next;
8 }
9 SelectPlan*selectPlan = SelectPlanInit(p,queryData->predicate);
10 p = PlanInit(selectPlan,PLAN_SELECT_CODE);
11 ProjectPlan*projectPlan = ProjectPlanInit(p,queryData->fields);
12 p = PlanInit(projectPlan,PLAN_PROJECT_CODE);
```

第三段代码来自 plan/ProjectPlan.c 的 ProjectPlanOpen。它先打开内部 Plan 得到下层 Scan，再用字段列表创建 ProjectScan，最后包装为统一的 Scan。

代码 9-6 ProjectPlanOpen 创建 ProjectScan

```
1 Scan* ProjectPlanOpen(void *data){
2     Plan*plan = (Plan*)data;
```

```

3   ProjectPlan * projectPlan = plan->planUnion.projectPlan;
4   Scan *s1 = projectPlan->p->open(projectPlan->p);
5
6   CList *fieldlist = CListInit(NULL, CStringListEquals, NULL);
7   Schema* schema = projectPlan->schema;
8   FieldNode *current = schema->fields;
9   while (current) {
10      CString *fldname = CStringCreateFromCString(current->fileName);
11      CListAppend(fieldlist, fldname);
12      current = current->next;
13  }
14  ProjectScan *projectScan = ProjectScanInit(s1, fieldlist);
15  Scan *scan = ScanInit(projectScan, SCAN_PROJECT_CODE);
16  return scan;
17 }

```

第四段代码来自 query/SelectScan.c 的 SelectScanNext。它持续推进内部 Scan，直到找到满足谓词的记录或扫描结束。

代码 9-7 SelectScanNext 过滤记录

```

1 bool SelectScanNext(void *data){
2     Scan*scan = (Scan*)data;
3     SelectScan *selectScan = scan->scanUnion.selectScan;
4     while (selectScan->s->next(selectScan->s)){
5         if(PredicateIsSatisfied(selectScan->predicate,selectScan->s)){
6             return true;
7         }
8     }
9     return false;
10 }

```

9.9 运行与观察

本章测试验证命令如下：

代码 9-8 build-plancheck 中验证 Plan/Scan 测试

```

1 mingw32-make -C build-plancheck PlanScanBasicTest -j8 SHELL=cmd.exe
2 ctest --test-dir build-plancheck --output-on-failure -R PlanScanBasicTest
3 ctest --test-dir build-plancheck --output-on-failure -R Plan

```

观察测试输出时，可以重点看以下问题：

1. PlannerCreateQueryPlan 是否能根据 SELECT SQL 返回非空 Plan。

2. 最外层 Plan 是否为 PLAN_PROJECT_CODE, 说明字段投影位于计划树最外层。
3. 计划树中是否能看到 ProjectPlan -> SelectPlan -> TablePlan 的基本结构。
4. plan->open(plan) 是否能返回非空 Scan。
5. 通过公开 Scan 接口是否只能读回 id = 1 的 wang, 而不会读出 id = 2 的 li。
6. 当前测试为什么不直接访问 ProjectScan 或 SelectScan 的内部字段: 这些类型在公开头文件中是不完整类型, 本章应通过统一 Scan 接口观察行为。

9.10 思考题

1. 为什么 DBMS_C 不直接执行 QueryData, 而要先构造 Plan?
2. Plan 和 Scan 的区别是什么? 如果把二者合并, 会失去哪些表达能力?
3. 为什么 ProjectPlan 通常位于计划树最外层?
4. ProductPlan 为什么可能导致记录数迅速增多?
5. 如果要优化多表查询顺序, 应该改进 Parser、Plan 还是 Scan? 为什么?

第 10 章 更新执行实验

10.1 实验目的

第 8 章已经观察了 SQL 文本如何被解析为结构化的 Data 对象，第 9 章进一步观察了 SELECT 查询如何从 QueryData 变成 Plan 和 Scan。本章只关注另一条主线：更新类 SQL 如何从解析结果进入 PlannerExecuteUpdate，再由 BasicUpdatePlanner 分发到具体执行函数。

本章的实验目标是理解 DBMS_C 中 CREATE TABLE、INSERT、UPDATE、DELETE、CREATE VIEW 和 CREATE INDEX 的命令级执行路径。这里的“更新”不是只指 SQL UPDATE，而是指所有会改变数据库状态或元数据目录的命令。本章不展开复杂事务恢复，也不重复查询计划细节；这些内容已经分别在第 5 章和第 9 章讨论。

10.2 更新执行在系统中的位置

更新执行位于 Parser 之后、元数据管理和记录访问之前。Parser 负责把命令解析为 CommandData；PlannerExecuteUpdate 根据命令类型做分发；BasicUpdatePlanner 根据不同命令调用元数据管理器或打开表扫描器执行修改。

图 10-1 展示了本章关注的主链路。可以看到，CREATE TABLE、CREATE VIEW、CREATE INDEX 主要走元数据登记路径，而 INSERT、UPDATE、DELETE 会打开表扫描并修改记录。

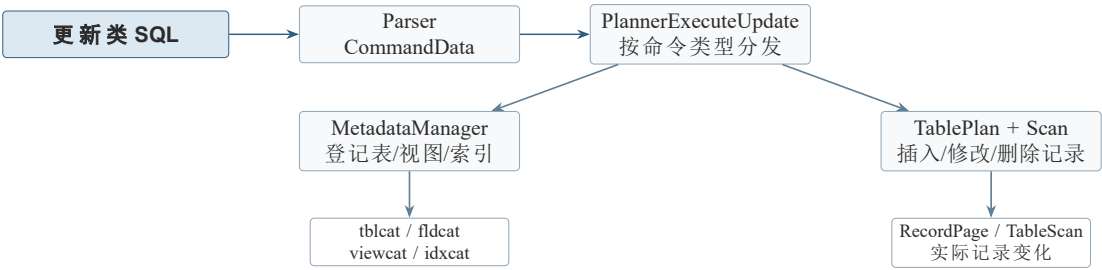


图 10-1 更新执行在 DBMS_C 主链路中的位置

10.3 相关源码文件

表 10-1 列出本章直接涉及的真实源码文件。阅读时建议先看 Planner.c 中的入口，再看 BasicUpdatePlanner.c 中每个 Execute 函数，最后回到 metadata 和 record 层观察结果如何落地。

表 10-1 更新执行相关源码文件

文件	本章关注点
plan/Planner.h, plan/Planner.c	更新执行入口 PlannerExecuteUpdate, 负责解析命令并按 CommandCode 分发。
plan/BasicUpdatePlanner.h, plan/BasicUpdatePlanner.c	CREATE TABLE、INSERT、UPDATE、DELETE、CREATE VIEW、CREATE INDEX 的具体高层执行逻辑。
parse/Parser.h, parse/Parser.c	ParserUpdateCmd 把更新类 SQL 转换为 CommandData。
parse/InsertData.h, parse/ModifyData.h, parse/DeleteData.h	INSERT、UPDATE、DELETE 的解析结果对象。
parse/CreateTableData.h, parse/CreateViewData.h, parse/CreateIndexData.h	CREATE TABLE、CREATE VIEW、CREATE INDEX 的解析结果对象。
metadata/MetadataManager.h, metadata/MetadataManager.c	表、视图、索引元数据登记和读取入口。
record/TableScan.h, record/TableScan.c	INSERT、UPDATE、DELETE 执行时使用的表扫描和记录修改接口。
query/Scan.h, query/Scan.c	统一 Scan 接口, BasicUpdatePlanner 通过该接口调用 insert、delete、setVal。
tx/Transaction.h, tx/Transaction.c	更新执行所在事务上下文, 最终提交时确认事务状态。

10.4 PlannerExecuteUpdate 与 BasicUpdatePlanner

PlannerExecuteUpdate 是更新类命令的统一入口。它先创建 Parser, 再调用 ParserUpdateCmd 得到 CommandData。随后根据 commandData->code 选择对应的 BasicUpdatePlannerExecute* 函数。

图 10-2 展示了这个分发关系。它不是复杂优化器, 而是一个清晰的命令路由器: 解析阶段已经识别出命令类型, 执行阶段只需要进入相应分支。

表 10-2 把命令类型和执行函数对应起来。本章测试会覆盖表中的所有基础命令。

10.5 各类更新命令的执行路径

CREATE TABLE、CREATE VIEW 和 CREATE INDEX 属于元数据登记路径。它们不会直接插入用户表记录, 而是修改系统目录表。例如 CREATE TABLE 会登记表名、字段名、字段类型和长度; CREATE VIEW 会登记视图名和视图定义; CREATE INDEX 会登记索引名、表名和字段名。

图 10-3 展示了这三类命令与系统目录的关系。这里的系统目录不是抽象概念, 而是由前面元数据章节讨论过的 tblcat、fldcat、viewcat 和 idxcat 支撑。

INSERT、UPDATE 和 DELETE 属于记录修改路径。它们会构造 TablePlan, 打开

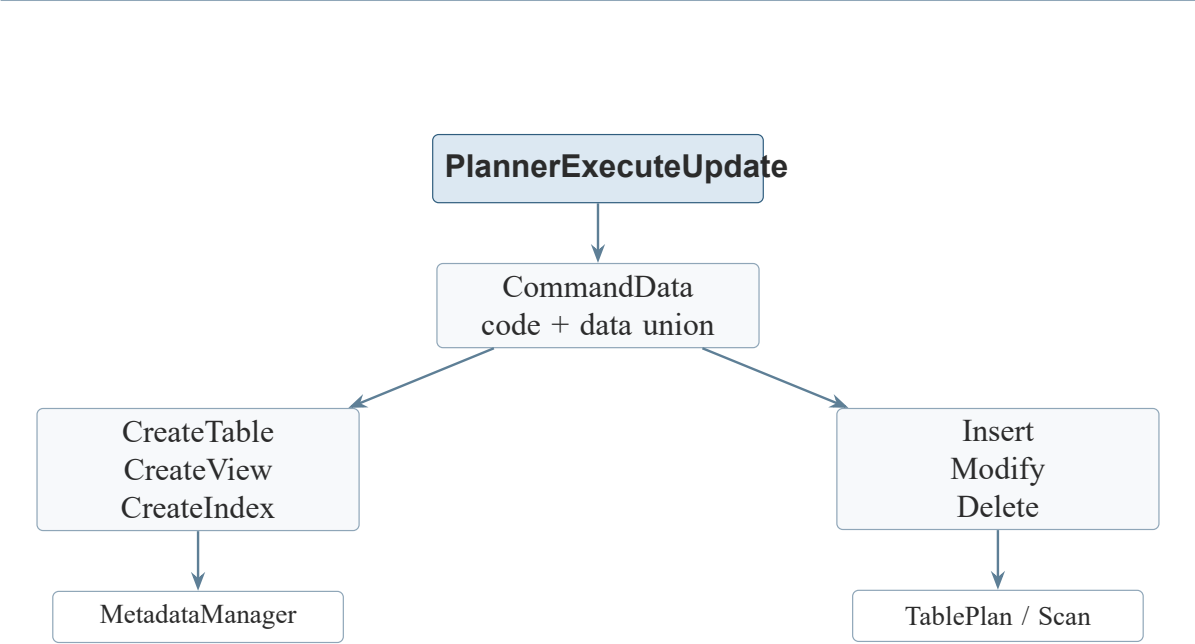


图 10-2 PlannerExecuteUpdate 的命令分发

表 10-2 CommandData 与 BasicUpdatePlanner 函数映射

命令类型	执行函数	主要动作
CMD_CREATE_TABLE	BasicUpdatePlannerExecuteCreateTable	调用 MetadataMgrCreateTable 登记表结构。
CMD_INSERT_DATA	BasicUpdatePlannerExecuteInsert	打开表扫描，插入新 slot，并按字段写入值。
CMD_MODIFY_DATA	BasicUpdatePlannerExecuteModify	使用 SelectPlan 定位匹配记录，计算新值并调用 setVal。
CMD_DELETE_DATA	BasicUpdatePlannerExecuteDelete	使用 SelectPlan 定位匹配记录，并调用 delete。
CMD_CREATE_VIEW	BasicUpdatePlannerExecuteCreateView	将 QueryData 转为视图定义字符串并登记到 viewcat。
CMD_CREATE_INDEX	BasicUpdatePlannerExecuteCreateIndex	调用 MetadataMgrCreateIndex 登记索引信息到 idxcat。

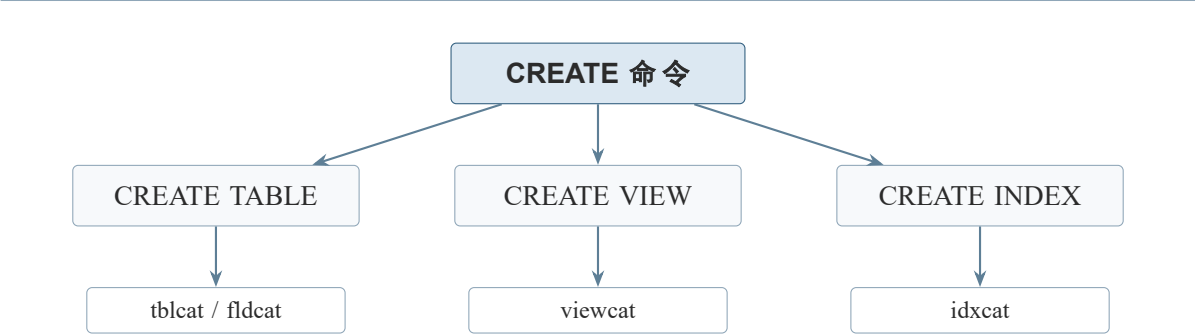


图 10-3 CREATE TABLE / VIEW / INDEX 的元数据登记路径

为 Scan，再通过统一 Scan 接口插入、设置或删除记录。图 10-4 展示了这三类命令的共同骨架。

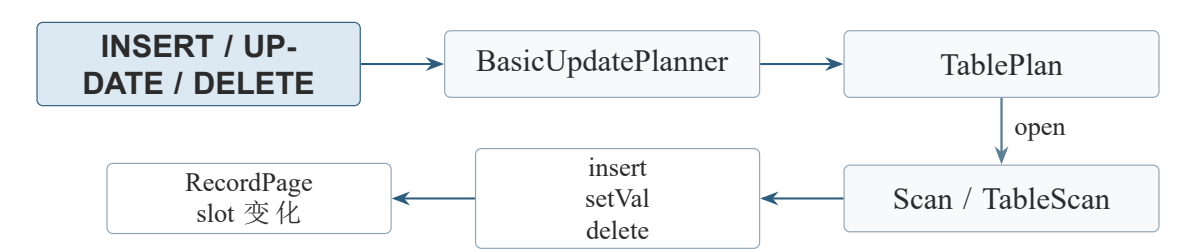


图 10-4 INSERT / UPDATE / DELETE 的记录修改链路

需要注意的是，UPDATE 和 DELETE 并不是直接遍历整表后立刻改动所有记录，而是在 TablePlan 外包一层 SelectPlan。因此 WHERE 谓词仍然由第 9 章讲过的 Scan 过滤机制负责，只是过滤出来的记录会被修改或删除。

10.6 配套测试

本章新增配套测试：

- test/plan/UpdateBasicTest.c

该测试使用独立测试数据库目录，目录名前缀为 update_basic_test_db，日志文件为 update_basic.log。每个用例都会创建 FileManager、LogManager、BufferManager、Transaction、MetadataManager 和 Planner，然后从 PlannerExecuteUpdate 入口执行 SQL 命令。测试不调用 BasicUpdatePlannerExecute* 的内部实现细节，而是用元数据接口和 TableScan 验证执行结果。

本章测试已在临时构建目录 build-updatecheck2 中验证通过。单独运行本章测试：

代码 10-1 运行 UpdateBasicTest

```
1 ctest --test-dir build-updatecheck2 --output-on-failure -R UpdateBasicTest
```

如需同时运行 Plan 和 Update 相关测试：

表 10-3 UpdateBasicTest 覆盖内容

测试函数	主要断言
test_update_planner_create_table_and_insert	断言 CREATE TABLE 返回 0；读回 Layout，断言 id/name 字段存在、类型和长度正确；执行两条 INSERT，断言返回值均为 1；用 TableScan 读回两条记录，断言 id=1,name=wang 和 id=2,name=li 存在；提交事务并断言状态为 TX_TRANSACTION_COMMIT。
test_update_planner_modify_and_delete	先建表并插入两条记录；执行 UPDATE，断言返回 1，且 id=1 的 name 变为 newwang；执行 DELETE，断言返回 1，且 id=2 不再出现；断言剩余记录数为 1；提交事务。
test_update_planner_create_view_and_index	执行 CREATE VIEW，断言可通过 MetadataMgrGetViewDef 读回视图定义；执行 CREATE INDEX，断言 MetadataManagerGetIndexInfo 返回一个索引信息，且字段 id 对应索引名为 idx_upd_id；提交事务。

代码 10-2 运行 Plan/Update 相关测试

```
1 ctest --test-dir build-updatecheck2 --output-on-failure -R "Update|Plan"
```

10.7 关键代码讲解

第一段代码来自 plan/Planner.c 的 PlannerExecuteUpdate。它体现了更新执行的统一入口：Parser 负责识别命令，Planner 根据 CommandCode 分发。

代码 10-3 PlannerExecuteUpdate 的命令分发

```
1 int PlannerExecuteUpdate(Planner*planner,CString *cmd,Transaction*transaction){
2     Parser*parser = ParserInit(CStringGetPtr(cmd));
3     CommandData *commandData = ParserUpdateCmd(parser);
4     CommandDataTrace(commandData);
5     CommandCode commandCode = commandData->code;
6     if(commandCode==CMD_INSERT_DATA){
7         return BasicUpdatePlannerExecuteInsert(planner->updatePlanner,
8                                                 commandData->data.insertData,
9                                                 transaction);
10    }else if(commandCode==CMD_DELETE_DATA){
11        return BasicUpdatePlannerExecuteDelete(planner->updatePlanner,
12                                                commandData->data.deleteData,
13                                                transaction);
14    }
15    /* 其他 CREATE / MODIFY 分支同样转给 BasicUpdatePlanner */
16    return 0;
17 }
```

第二段代码来自 plan/BasicUpdatePlanner.c 的 INSERT 执行函数。它先为目标表创建 TablePlan，打开为 Scan，然后插入一条记录并逐字段写入值。

代码 10-4 BasicUpdatePlannerExecuteInsert 的核心逻辑

```
1 TablePlan *tablePlan = TablePlanInit(transaction,data->tblname,
2                                     basicUpdatePlanner->metadataMgr);
3 Plan *plan = PlanInit(tablePlan,PLAN_TABLE_CODE);
4 Scan *scan = plan->open(plan);
5 scan->insert(scan);
6 CListNode *fldHead = data->flds->head;
7 CListNode *valHead = data->vals->head;
8 while (fldHead){
9     Constant *val = ((Constant *)valHead->data);
10    CString*fldname = ((CString *)fldHead->data);
11    scan->setVal(scan,fldname,val);
12    fldHead = fldHead->next;
```

```

13     valHead = valHead->next;
14 }

```

第三段代码来自 UPDATE 执行函数。它把目标表包装为 SelectPlan，只对满足谓词的记录计算新值并调用 setVal。

代码 10-5 BasicUpdatePlannerExecuteModify 的核心逻辑

```

1 TablePlan *tablePlan = TablePlanInit(transaction,data->tblname,
2                                     basicUpdatePlanner->metadataMgr);
3 Plan *plan = PlanInit(tablePlan,PLAN_TABLE_CODE);
4 SelectPlan *selectPlan = SelectPlanInit(plan,data->predicate);
5 plan = PlanInit(selectPlan,PLAN_SELECT_CODE);
6 Scan *scan = plan->open(plan);
7 int count = 0;
8 while (scan->next(scan)){
9     Constant *val = ExpressionEvaluate(data->newVal,scan);
10    scan->setVal(scan,data->fldname,val);
11    count++;
12 }

```

第四段代码来自 CREATE VIEW 和 CREATE INDEX 的执行逻辑。它们不直接修改用户表记录，而是调用元数据管理器登记定义。

代码 10-6 CREATE VIEW 和 CREATE INDEX 的元数据登记

```

1 int BasicUpdatePlannerExecuteCreateView(BasicUpdatePlanner *planner,
2                                         CreateViewData *data,
3                                         Transaction*transaction){
4     char *viewDefStr = QueryDataToString(data->queryData);
5     CString *viewDef = CStringCreateFromCStr(viewDefStr);
6     MetadataMgrCreateView(planner->metadataMgr,data->viewName,
7                           viewDef,transaction);
8     CStringDestroy(viewDef);
9     return 0;
10 }
11
12 int BasicUpdatePlannerExecuteCreateIndex(BasicUpdatePlanner *planner,
13                                         CreateIndexData *data,
14                                         Transaction*transaction){
15     MetadataMgrCreateIndex(planner->metadataMgr,data->idxname,
16                           data->tblname,data->fldname,transaction);
17     return 0;
18 }

```

10.8 运行与观察

本章验证使用临时构建目录 `build-updatecheck2`，没有使用旧 `build` 目录，也没有同步任何 CMake 生成的 `.tmp` 文件。测试通过后，可以重点观察以下问题：

- `CREATE TABLE` 是否能够通过 `MetadataMgrGetLayout` 读回字段类型和长度。
- `INSERT` 是否通过返回值和 `TableScan` 读回结果共同确认记录写入。
- `UPDATE` 是否只修改满足 `WHERE` 谓词的记录。
- `DELETE` 是否只删除满足 `WHERE` 谓词的记录。
- `CREATE VIEW` 是否把 `QueryData` 转成可读回的视图定义字符串。
- `CREATE INDEX` 是否在 `idxcat` 中登记索引名、表名和字段名。

这里还可以观察一个容易忽略的实现边界：系统目录字段有固定长度。本章测试使用较短索引名 `idx_upd_id`，是为了符合当前 `idxcat` 字段长度，避免测试因为目录字段截断而失败。这个限制来自当前实现，本章不通过修改元数据字段长度来绕过它。

10.9 思考题

1. 为什么 `DBMS_C` 把 `SELECT` 查询和更新类命令分成 `CreateQueryPlan` 与 `ExecuteUpdate` 两条入口？
2. `INSERT`、`UPDATE`、`DELETE` 都会打开表扫描器，它们对 `Scan` 接口的使用有什么相同点和不同点？
3. `CREATE TABLE`、`CREATE VIEW`、`CREATE INDEX` 为什么可以看作“修改系统目录”的操作？
4. `UPDATE` 和 `DELETE` 为什么需要先经过 `SelectPlan`？
5. 如果系统目录字段长度过短，会对表名、字段名、索引名产生什么影响？

第 11 章 SQL 执行过程 Trace 可视化实验

11.1 实验目的

本章通过 DBMS_C 已有的 Trace 模式观察 SQL 在系统内部的执行链路。前面第 8 章已经说明 SQL 如何被解析为 QueryData 或各类更新命令 Data 对象，第 9 章说明 SELECT 如何生成 Plan 并打开为 Scan，第 10 章说明更新命令如何进入 PlannerExecuteUpdate 和 BasicUpdatePlanner。Trace 模式把这些分散在源码中的过程以命令行文本形式输出出来，帮助读者把解析、计划、扫描、更新、事务和结果输出连成一条可观察的主线。

本章是人工验证型实验，不新增自动 CTest。Trace 输出依赖交互式 CLI、标准输入和进程退出路径；此前尝试自动化运行时出现挂住和资源占用风险，因此本章选择更稳妥的人工演示方式。

11.2 Trace 模式定位

Trace 是 CLI 文本过程可视化，不是图形界面。它默认关闭，只有通过启动参数或交互命令开启后才会输出额外的 [TRACE_*] 行。Trace 的作用是观察执行过程，不参与 SQL 语义判断，也不应该改变查询结果、更新结果或事务行为。

图 11-1 展示了 Trace 在 DBMS_C 执行链路中的位置。Trace 不是新的执行层，而是分布在关键边界处的观察点：Parser 生成数据对象后输出解析摘要，Planner 生成 Plan 后输出计划树，Plan 打开 Scan 后输出扫描链，更新和事务入口输出命令级过程。

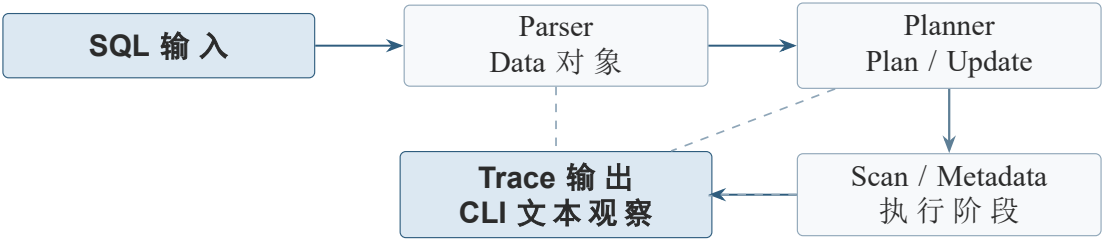


图 11-1 Trace 在 DBMS_C 执行链路中的位置

11.3 开启与关闭方式

Trace 可以在启动时开启，也可以在 CLI 运行过程中切换。启动时开启使用：

代码 11-1 启动时开启 Trace

```
1 bin\NewDBMS.exe --trace
```

进入 SQL 提示符后，可以使用以下元命令：

代码 11-2 交互式 Trace 命令

```
1 trace on;
2 trace off;
3 trace;
```

--trace 表示程序启动时开启 Trace; trace on; 表示运行过程中打开 Trace; trace off; 表示关闭 Trace; trace; 用于查看当前状态。普通模式下不会输出 Trace 行，因此普通 SQL 输出不受影响。

11.4 Trace 标签说明

Trace 标签由 DBTraceLog 统一输出，格式为 [TRACE_ 阶段] 消息。表 11-1 列出当前源码中实际使用的标签。

表 11-1 Trace 标签说明

标签	含义
[TRACE_PARSE]	解析阶段摘要，例如字段列表、表名列表、谓词、更新命令参数。
[TRACE_PLAN]	SELECT 查询计划树，例如 ProjectPlan、SelectPlan、TablePlan。
[TRACE_SCAN]	运行期扫描链，例如 ProjectScan、SelectScan、TableScan。
[TRACE_OUTPUT]	输出阶段，例如渲染 SELECT 结果、结果行数、命令完成信息。
[TRACE_UPDATE]	更新命令类型，例如 CREATE TABLE、INSERT、UPDATE、DELETE。
[TRACE_PLANNER]	更新命令被分发到具体 BasicUpdatePlanner 执行函数。
[TRACE_METADATA]	元数据登记动作，例如登记表结构、视图定义、索引信息。
[TRACE_RECORD]	记录级摘要动作，例如插入、更新或删除匹配记录。
[TRACE_TX]	事务动作，例如 COMMIT、ROLLBACK、事务编号和提交/回滚步骤。

11.5 SELECT 查询 Trace 示例

下面的示例用于观察 SELECT 主链路。为了避免已有数据库目录中的旧数据影响演示，建议每次使用新的表名。

代码 11-3 SELECT Trace 示例 SQL

```
1 CREATE TABLE student_trace_demo(id INT, name VARCHAR(20));
2 INSERT INTO student_trace_demo(id, name) VALUES(1, 'wang');
3 INSERT INTO student_trace_demo(id, name) VALUES(2, 'li');
4 SELECT id, name FROM student_trace_demo WHERE id = 1;
```

开启 Trace 后，可以重点观察如下片段：

代码 11-4 SELECT Trace 关键输出片段

```
1 [TRACE_PARSE] QueryData
2 [TRACE_PARSE]   fields: id, name
3 [TRACE_PARSE]   tables: student_trace_demo
4 [TRACE_PLAN] Plan tree
5 [TRACE_PLAN]   ProjectPlan fields=id, name
6 [TRACE_SCAN] Scan chain
7 [TRACE_SCAN]   ProjectScan fields=id, name
8 [TRACE_OUTPUT] Rendering SELECT result rows
9 [TRACE_OUTPUT] result rows: 1
```

图 11-2 展示这些 Trace 行与前面章节内容的对应关系。QueryData 对应第 8 章 SQL 解析；ProjectPlan / SelectPlan / TablePlan 对应第 9 章查询计划；ProjectScan / SelectScan / TableScan 对应运行期扫描链；result rows 则对应最终输出阶段。

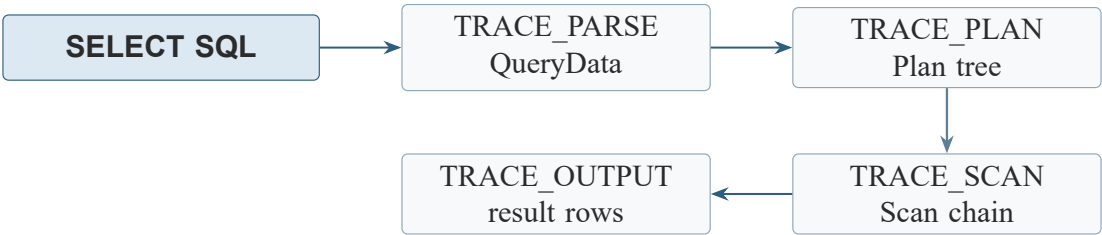


图 11-2 SELECT Trace 阶段链路

11.6 更新命令 Trace 示例

更新命令 Trace 主要观察命令级过程，而不是深入 Buffer、File、Log 或 Recovery 的底层细节。CREATE TABLE 会显示命令类型、表名、字段摘要、Planner 分发和元数据登记；INSERT 会显示命令类型、表名和值摘要、打开表扫描、插入记录和事务层写入；UPDATE 和 DELETE 会显示谓词定位、记录修改或删除以及事务层摘要。

代码 11-5 更新命令 Trace 片段

```
1 [TRACE_UPDATE] CREATE TABLE command
2 [TRACE_PARSE]   table: student_trace_demo
3 [TRACE_METADATA] register table schema in catalog
4
5 [TRACE_UPDATE] INSERT command
6 [TRACE_PLANNER] dispatch to BasicUpdatePlannerExecuteInsert
7 [TRACE_SCAN] open table scan for student_trace_demo
8 [TRACE_RECORD] insert new record
9
10 [TRACE_UPDATE] UPDATE command
11 [TRACE_RECORD] update matched record
12
```

```
13 [TRACE_UPDATE] DELETE command
14 [TRACE_RECORD] delete matched record
```

这些输出与第 10 章的 `PlannerExecuteUpdate` 和 `BasicUpdatePlannerExecute*` 直接对应。Trace 只是把分发和执行摘要打印出来，不改变命令返回的 `affected rows`，也不替代真正的模块测试。

11.7 事务命令 Trace 示例

事务命令 Trace 关注 COMMIT 和 ROLLBACK 的命令级过程。COMMIT 会输出提交命令、事务编号、flush modified buffers、write commit log 和 transaction committed; ROLLBACK 会输出回滚命令、事务编号、rollback transaction changes if any 和 transaction rolled back。

代码 11-6 事务 Trace 片段

```
1 [TRACE_TX] COMMIT command
2 [TRACE_TX] transaction: 3
3 [TRACE_TX] flush modified buffers
4 [TRACE_TX] write commit log
5 [TRACE_OUTPUT] transaction committed
6
7 [TRACE_TX] ROLLBACK command
8 [TRACE_TX] transaction: 4
9 [TRACE_TX] rollback transaction changes if any
10 [TRACE_OUTPUT] transaction rolled back
```

需要注意，`exit` 会进入当前程序已有的关闭路径，关闭前会提交当前事务。因此 Trace 开启时，即使用户只是输入 `exit`，也可能看到退出阶段的 COMMIT Trace。这是现有事务关闭逻辑的一部分，不是额外的 Trace 行为。

11.8 人工验证步骤

本章不运行自动测试，而是采用人工验证。进入项目目录后启动程序：

代码 11-7 启动 Trace 演示

```
1 cd D:\code\DBMS\NewDBMS\DBMS_C
2 .\bin\NewDBMS.exe --trace
```

然后在 SQL 提示符中输入以下命令：

代码 11-8 人工验证 SQL

```
1 CREATE TABLE student_trace_demo_001(id INT, name VARCHAR(20));
2 INSERT INTO student_trace_demo_001(id, name) VALUES(1, 'wang');
3 INSERT INTO student_trace_demo_001(id, name) VALUES(2, 'li');
4 SELECT id, name FROM student_trace_demo_001 WHERE id = 1;
5 UPDATE student_trace_demo_001 SET name = 'newwang' WHERE id = 1;
6 SELECT id, name FROM student_trace_demo_001 WHERE id = 1;
7 DELETE FROM student_trace_demo_001 WHERE id = 2;
8 SELECT id, name FROM student_trace_demo_001 WHERE id = 2;
9 commit;
10 exit;
```

预期观察点如下：SELECT 语句会出现 Parse、Plan、Scan 和 Output Trace；UPDATE 与 DELETE 会出现更新命令和记录修改 Trace；删除后再次查询 id = 2 时，可以观察 result rows: 0；commit；会出现事务 Trace。

表 11-2 人工验证覆盖表

验证内容	观察点
Trace 开关	使用 --trace 启动，或在交互中使用 trace on;、trace off;、trace;。
SELECT 主链路	观察 TRACE_PARSE、TRACE_PLAN、TRACE_SCAN 和 TRACE_OUTPUT。
更新执行	观察 TRACE_UPDATE、TRACE_PLANNER、TRACE_METADATA、TRACE_RECORD。
空结果查询	删除记录后再次查询，观察 result rows: 0。
事务命令	使用 commit; 或 rollback;，观察 TRACE_TX。

11.9 演示注意事项

当前 main.c 使用固定数据库名 Show2。系统启动时会恢复已有数据库，因此重复运行同一个 demo 表名可能读到旧数据或旧记录。演示时建议每次使用新的表名，例如 student_trace_demo_001、student_trace_demo_002，或者在独立工作目录中运行程序。这不是 Trace bug，而是已有数据库恢复行为造成的演示现象。

还要注意，当前系统目录中的名称字段有固定长度。如果人工演示时遇到长表名、长索引名被截断的现象，可以改用更短的表名继续观察 Trace 主链路。

11.10 配套验证说明

本章不新增自动 CTest。原因有三点：第一，Trace 属于 CLI 交互式输出，自动化测试需要同时处理输入、输出捕获、工作目录和退出路径；第二，自动启动 NewDBMS.exe 曾出现挂住和资源占用风险；第三，第 8、9、10 章已经分别通过 ParseBasicTest、PlanScanBasicTest、UpdateBasicTest 验证了解析、查询计划/扫描和更新执行核心链路。

因此，本章采用人工验证方式。读者完成演示后，只需要确认 Trace 标签和 SQL 执行链路能够对应起来即可。

11.11 关键代码讲解

第一段代码来自 `trace/DBTrace.c`。Trace 默认关闭，只有启用后 `DBTraceLog` 才会输出 `[TRACE_*]` 行。

代码 11-9 DBTraceLog 的输出封装

```
1 void DBTraceLog(const char *stage, const char *message) {
2     if (!g_trace_enabled) {
3         return;
4     }
5
6     printf("[TRACE_%s] ", stage ? stage : "GENERAL");
7     for (int i = 0; i < g_trace_indent; i++) {
8         printf(" ");
9     }
10    printf("%s\n", message ? message : "");
11 }
```

第二段代码来自 `main.c`。启动参数 `--trace` 会在程序启动时打开 Trace；交互命令 `trace on/off/trace` 则用于运行时切换或查看状态。

代码 11-10 main.c 中的 Trace 开关

```
1 for (int i = 1; i < argc; i++) {
2     if (strcmp(argv[i], "--trace") == 0) {
3         DBTraceEnable();
4     }
5 }
6
7 /* 交互命令分支 */
8 DBTraceEnable();
9 printf("Trace mode enabled.\n");
10 DBTraceDisable();
11 printf("Trace mode disabled.\n");
12 printf("Trace mode is %s.\n", DBTraceIsEnabled() ? "on" : "off");
```

第三段代码来自 `plan/Planner.c` 和 `plan/BasicQueryPlanner.c`。SELECT 查询在解析后输出 `QueryData`，生成最终 `ProjectPlan` 后输出 `Plan tree`。

代码 11-11 查询链路中的 Trace 调用

```
1 QueryData*data = ParserQuery(parser);
2 QueryDataTrace(data);
3 return BasicQueryPlannerCreatPlan(planner->queryPlanner,data,transaction);
4
5 ProjectPlan*projectPlan = ProjectPlanInit(p,queryData->fields);
6 p = PlanInit(projectPlan,PLAN_PROJECT_CODE);
7 PlanTraceTree(p);
```

第四段代码来自 plan/BasicUpdatePlanner.c 和 tx/Transaction.c。更新命令和事务命令在入口处输出命令级过程 Trace。

代码 11-12 更新与事务链路中的 Trace 调用

```
1 int BasicUpdatePlannerExecuteInsert(BasicUpdatePlanner *planner,
2                                     InsertData *data,
3                                     Transaction*transaction){
4     UpdateExecutionTraceInsert(data);
5     /* 后续打开 TablePlan / Scan 并插入记录 */
6 }
7
8 void TransactionCommit(Transaction*transaction){
9     TransactionTraceCommit(transaction);
10    RecoveryCommit(transaction->recoveryManager);
11    printf("transaction %d commit\n",transaction->txNum);
12 }
```

11.12 运行与观察

运行时先观察普通 SQL 输出，再观察额外的 Trace 行。普通输出包括查询表格、命令影响行数和事务提交信息；Trace 输出则以 [TRACE_*] 开头，用于解释这些普通输出背后的内部过程。

表 11-3 普通模式与 Trace 模式对照

模式	输出内容	适用场景
普通模式	只显示查询结果、命令执行结果和事务提示。	日常运行、验证 SQL 结果。
Trace 模式	在普通输出之外增加解析、计划、扫描、更新、事务和输出阶段摘要。	教学、调试、演示执行链路。

观察时可以按标签回到对应章节：看到 TRACE_PARSE 时回看第 8 章，看到 TRACE_PLAN 和 TRACE_SCAN 时回看第 9 章，看到 TRACE_UPDATE 和 TRACE_PLANNER 时

回看第 10 章，看到 TRACE_TX 时回看第 5 章。

11.13 思考题

1. Trace 为什么要默认关闭？
2. Trace 与 LogManager 管理的数据库日志有什么区别？
3. 为什么 Trace 不应该改变 SQL 的执行结果？
4. 如果未来要做 Web 或 GUI 可视化，需要在 Trace 信息中增加哪些结构化字段？

第 12 章 综合实验

12.1 实验目的

TODO: 本章将在补充并通过对应 CTest 后完善。

12.2 相关源码文件

TODO

12.3 配套测试

TODO: 先补充测试，再撰写本章内容。

12.4 实验内容

TODO

12.5 思考题

TODO

后续维护原则

后续扩展本指导书时，所有函数名、结构体名、目录路径、测试文件路径和运行结果都必须来自当前仓库的真实状态。若某一章尚未补充并通过对应测试，只能保留 TODO 或说明“后续补充”，不能写入未验证的测试结论。